autrainer

A Modular and Extensible Deep Learning Toolkit for Computer Audition Tasks.

autrainer is built on top of PyTorch and Hydra, offering a modular and extensible way to perform reproducible deep learning experiments for computer audition tasks using YAML configuration files and the command line.

Installation

To install *autrainer*, first ensure that PyTorch (along with torchvision and torchaudio) version 2.0 or higher is installed. For installation instructions, refer to the PyTorch website.

It is recommended to install *autrainer* within a virtual environment. To create a new virtual environment, refer to the Python venv documentation.

Next, install *autrainer* using *pip*.

```
pip install autrainer
```

The following optional dependencies can be installed to enable additional features:

- `latex` for LaTeX plotting (requires a LaTeX installation).
- `mlflow` for MLflow logging.
- `tensorboard` for TensorBoard logging.
- `opensmile` for audio feature extraction with openSMILE.
- `albumentations` for image augmentations with Albumentations.
- `audiomentations` for audio augmentations with audiomentations.
- `torch-audiomentations` for audio augmentations with torch-audiomentations.

```
pip install autrainer[latex]
pip install autrainer[mlflow]
pip install autrainer[tensorboard]
pip install autrainer[opensmile]
pip install autrainer[albumentations]
pip install autrainer[audiomentations]
pip install autrainer[torch-audiomentations]
```

To install *autrainer* with all optional dependencies, use the following command:

```
pip install autrainer[all]
```

To install *autrainer* from source, refer to the contribution guide.

Next Steps

To get started using *autrainer*, the quickstart guide outlines the creation of a simple training configuration and Tutorials provide examples for implementing custom modules including their configurations.

For a complete list of available CLI commands, refer to the CLI reference or the CLI wrapper.

Overview

Quickstart

The following quickstart guide provides a short introduction to *autrainer* and the creation of simple training experiments.

First Experiment

To get started, create a new directory and navigate to it:

```
mkdir autrainer_example && cd autrainer_example
```

Next, create a new empty *autrainer* project using the following configuration management CLI command:

```
autrainer create --empty
```

Alternatively, use the following configuration management CLI wrapper function:

```
import autrainer.cli # the import is omitted in the following examples for brevity

autrainer.cli.create(empty=True)
```

This will create the configuration directory structure and the main configuration (`conf/config.yaml`) file with default values:

conf/config.yaml

```yaml
defaults:
  - _autrainer_
  - _self_

results_dir: results
experiment_id: default
iterations: 5

hydra:
  sweeper:
    params:
      +seed: 1
      +batch_size: 32
      +learning_rate: 0.001
      dataset: ToyTabular-C
      model: ToyFFNN
      optimizer: Adam
```

Now, run the following training command to train the model:

```
autrainer train
```

Alternatively, use the following training CLI wrapper function:

```
autrainer.cli.train() # the train function is omitted in the following examples for brevity
```

This will train the default `ToyFFNN` feed-forward neural network (`FFNN`) on the default `ToyTabular-C` classification dataset with tabular data (`ToyDataset`) and output the training results to the `results/default/` directory.

Custom Model Configuration

The first experiment uses the default `ToyFFNN` model with the following configuration having 2 hidden layers:

conf/model/ToyFFNN.yaml

```yaml
id: ToyFFNN
_target_: autrainer.models.FFNN
input_size: 64
hidden_size: 64
num_layers: 2

transform:
  type: tabular
```

To create another configuration for the `FFNN` model with 3 hidden layers, create a new configuration file in the `conf/model/` directory:

conf/model/Three-Layer-FFNN.yaml

```yaml
id: Three-Layer-FFNN
_target_: autrainer.models.FFNN
input_size: 64
hidden_size: 64
num_layers: 3 # 3 hidden layers

transform:
  type: tabular
```

Next, update the main configuration (`conf/config.yaml`) file to use the new model configuration:

conf/config.yaml

```yaml
defaults:
  - _autrainer_
  - _self_

results_dir: results
experiment_id: default
iterations: 5

hydra:
  sweeper:
    params:
      +seed: 1
      +batch_size: 32
      +learning_rate: 0.001
      dataset: ToyTabular-C
      model: Three-Layer-FFNN # 3 hidden layers
      optimizer: Adam
```

Now, run the following training command to train the model with 3 hidden layers:

```
autrainer train
```

Grid Search Configuration

To perform a grid search over multiple configurations defined in the `params`, update the main configuration (`conf/config.yaml`) to include multiple values separated by a comma.

The following configuration performs a grid search over the default `FFNN` model with 2 and 3 hidden layers as well as 3 different seeds:

conf/config.yaml

```yaml
defaults:
  - _autrainer_
  - _self_

results_dir: results
experiment_id: default
iterations: 5

hydra:
  sweeper:
    params:
      +seed: 1, 2, 3 # 3 seeds to compare
      +batch_size: 32
      +learning_rate: 0.001
      dataset: ToyTabular-C
      model: ToyFFNN, Three-Layer-FFNN # 2 models to compare
      optimizer: Adam
```

Now, run the following training command to train the models with 2 and 3 hidden layers and 3 different seeds:

```
autrainer train
```

By default, a grid search is performed sequentially. Hydra allows the use of different launcher plugins to perform parallel grid searches.

> **Note**
>
> If a run already exists in the same experiment and has been completed successfully, then it will be skipped. This may be the case for both the default and custom model configurations with seed 1 if they have already been trained in the previous examples.

To compare the results of the individual runs as well as averaged across seeds, run the following postprocessing command:

```
autrainer postprocess results default --aggregate seed
```

Alternatively, use the following postprocessing CLI wrapper function:

```
autrainer.cli.postprocess(
    results_dir="results",
    experiment_id="default",
    aggregate=[["seed"]],
)
```

Spectrogram Classification

To train a `Cnn10` model on an audio dataset such as `DCASE2016Task1`, update the main configuration (`conf/config.yaml`) file:

conf/config.yaml

```yaml
defaults:
  - _autrainer_
  - _self_

results_dir: results
experiment_id: spectrogram
iterations: 5

hydra:
  sweeper:
    params:
      +seed: 1
      +batch_size: 32
      +learning_rate: 0.001
      dataset: DCASE2016Task1-32k
      model: Cnn10-32k-T
      optimizer: Adam
```

The following configuration management command is used to discover all available default configurations for the `Cnn10` model:

```bash
autrainer list model --pattern=Cnn10*
```

Alternatively, use the following configuration management CLI wrapper function:

```python
autrainer.cli.list("model", pattern="Cnn10*")
```

For the `Cnn10` model, the following configuration is used:

conf/model/Cnn10-32k-T.yaml

```yaml
id: Cnn10-32k-T
_target_: autrainer.models.Cnn10
transfer: https://zenodo.org/records/3987831/files/Cnn10_mAP%3D0.380.pth

transform:
  type: grayscale
  base:
    - autrainer.transforms.Normalize: null
```

The ending `32k-T` indicates that the model uses transfer learning and has been pretrained with a sample rate of 32 kHz.

> **Tip**
>
> To discover all available default configurations for e.g., different models, the configuration management CLI, the configuration management CLI wrapper, and the models documentation can be used.

For the `DCASE2016Task1` dataset, the following configuration is used:

conf/dataset/DCASE2016Task1-32k.yaml

```yaml
id: DCASE2016Task1-32k
_target_: autrainer.datasets.DCASE2016Task1

fold: 1
```

```yaml

path: data/DCASE2016
features_subdir: log_mel_32k
index_column: filename
target_column: scene_label
file_type: npy
file_handler: autrainer.datasets.utils.NumpyFileHandler

criterion: autrainer.criterions.BalancedCrossEntropyLoss
metrics:
  - autrainer.metrics.Accuracy
  - autrainer.metrics.UAR
  - autrainer.metrics.F1
tracking_metric: autrainer.metrics.Accuracy

transform:
  type: grayscale
```

The ending `32k` indicates that the dataset has a sample rate of 32 kHz and provides log-Mel spectrograms instead of raw audio.

To avoid race conditions when using Launcher Plugins that may run multiple training jobs in parallel, the following preprocessing command is used to fetch and download the model weights and the raw audio files of the dataset:

```
autrainer fetch
```

Alternatively, use the following preprocessing CLI wrapper function:

```
autrainer.cli.fetch()
```

As the dataset uses log-Mel spectrograms instead of the raw audio files downloaded in the previous step, the following preprocessing command is used to preprocess and extract the features from the raw audio files:

```
autrainer preprocess
```

Alternatively, use the following preprocessing CLI wrapper function:

```
autrainer.cli.preprocess()
```

Now, run the following training command to train the model on the audio dataset:

```
autrainer train
```

Overriding Configurations

All *autrainer* default configurations can be easily overridden by creating a new configuration file in the corresponding directory with the same name as the default configuration. The new configuration file will be used instead of the default configuration.

To override the default `path` the `DCASE2016Task1` dataset stores files in, the following configuration management command is used to locally save the default configuration:

```
autrainer show dataset DCASE2016Task1-32k --save
```

Alternatively, use the following configuration management CLI wrapper function:

```
autrainer.cli.show("dataset", "DCASE2016Task1-32k", save=True)
```

This will save the default configuration to the `conf/dataset/DCASE2016Task1-32k.yaml` file, which can be edited to override the `path`:

conf/dataset/DCASE2016Task1-32k.yaml

```yaml
id: DCASE2016Task1-32k
_target_: autrainer.datasets.DCASE2016Task1

fold: 1

path: /some/custom/path # modify the default save path
features_subdir: log_mel_32k
index_column: filename
target_column: scene_label
file_type: npy
file_handler: autrainer.datasets.utils.NumpyFileHandler

criterion: autrainer.criterions.BalancedCrossEntropyLoss
metrics:
  - autrainer.metrics.Accuracy
  - autrainer.metrics.UAR
  - autrainer.metrics.F1
tracking_metric: autrainer.metrics.Accuracy

transform:
  type: grayscale
```

Training Duration & Step-based Training

By default, *autrainer* uses epoch-based training, where the `iterations` correspond to the number of epochs. To change the training duration of the spectrogram classification model, increase the number of `iterations` in the main configuration (`conf/config.yaml`) file.

However, to use step-based training instead of epoch-based training, set the `training_type` to `step`.

The following configuration trains the spectrogram classification model for a total of 1000 steps with step-based training, evaluating every 100 steps, saving the states every 200 steps, and without displaying a progress bar:

conf/config.yaml

```yaml
defaults:
  - _autrainer_
  - _self_

results_dir: results
experiment_id: spectrogram_step

training_type: step
iterations: 1000
eval_frequency: 100
save_frequency: 200
progress_bar: false

hydra:
  sweeper:
    params:
      +seed: 1
      +batch_size: 32
      +learning_rate: 0.001
      dataset: DCASE2016Task1-32k
      model: Cnn10-32k-T
      optimizer: Adam
```

Now, run the following training command to train the model on the audio dataset for 1000 steps:

```
autrainer train
```

Filtering Configurations

By default, *autrainer* filters out any configurations that have already been trained and exist in the same experiment using the hydra-filter-sweeper plugin with the following `filters` that are implicitly set in the _autrainer_.yaml defaults file:

conf/config.yaml

```yaml
  filters:
    - type: exists
      path: metrics.csv
```

To filter out unwanted configurations and exclude them from training, the hydra-filter-sweeper plugin can be used as the Hydra sweeper plugin. `hydra-filter-sweeper` allows to specify a list of `filters` to exclude configurations based on their attributes.

The following configuration expands the grid search configuration and adds a filter that excludes any seed greater than 2 for the `Three-Layer-FFNN` model:

conf/config.yaml

```yaml
defaults:
  - _autrainer_
  - _self_

results_dir: results
experiment_id: default
iterations: 5

hydra:
  sweeper:
    params:
      +seed: 1, 2, 3
      +batch_size: 32
      +learning_rate: 0.001
      dataset: ToyTabular-C
      model: ToyFFNN, Three-Layer-FFNN
      optimizer: Adam
    filters:
      - type: exists
        path: metrics.csv
      - type: expr
        expr: model.id == "Three-Layer-FFNN" and seed > 2
```

Note

If the `filters` attribute is overridden in the main configuration (`conf/config.yaml`) file, then the default filters are not applied. To still filter out configurations that have already been trained, the following default filter should still be included:

conf/config.yaml

```yaml
  filters:
    - type: exists
      path: metrics.csv
```

Now, run the following training command to train the `ToyFFNN` with 3 seeds and the `Three-Layer-FFNN` with 2 seeds:

```
autrainer train
```

Next Steps

For more information on creating configurations, refer to the Hydra configurations as well as the Hydra documentation.

To create custom implementations alongside configurations, refer to the tutorials.

Tutorials

autrainer is designed to be flexible and extensible, allowing for the creation of custom ...

- models
- datasets (including metrics, criterions, file handlers, target transforms, and advanced data pipelines)
- optimizers
- schedulers
- transforms (including preprocessing transforms and online transforms)
- augmentations
- loggers

For each, a tutorial is provided below to demonstrate their implementation and configuration.

For the following tutorials, all python files should be placed in the project root directory and all configuration files should be placed in the corresponding subdirectories of the `conf/` directory.

Custom Models

To create a custom model, inherit from `AbstractModel` and implement the `forward()` and `embeddings()` methods. All arguments of the constructor have to be assigned to a variable with the same name, as `AbstractModel` inherits from audobject.

For example, the following model is a simple CNN that takes a spectrogram as input and has a variable number of hidden CNN layers with a different number of filters each:

spectrogram_cnn.py

```python
from typing import List

import torch

from autrainer.models import AbstractModel


class SpectrogramCNN(AbstractModel):
    def __init__(self, output_dim: int, hidden_dims: List[int]) -> None:
        """Spectrogram CNN model with a variable number of hidden CNN layers.

        Args:
            output_dim: Output dimension of the model.
            hidden_dims: List of hidden dimensions for the CNN layers.
        """
        super().__init__(output_dim, None)  # no transfer learning
        self.hidden_dims = hidden_dims
        layers = []
        input_dim = 1
        for hidden_dim in self.hidden_dims:
            layers.extend(
                [
                    torch.nn.Conv2d(input_dim, hidden_dim, (3, 3), 1),
                    torch.nn.ReLU(),
                    torch.nn.MaxPool2d((2, 2)),
```

```
                ]
            )
            input_dim = hidden_dim
        layers.extend(
            [
                torch.nn.AdaptiveAvgPool2d((1, 1)),
                torch.nn.Flatten(),
            ]
        )
        self.backbone = torch.nn.Sequential(*layers)
        self.classifier = torch.nn.Linear(self.hidden_dims[-1], output_dim)

    def embeddings(self, features: torch.Tensor) -> torch.Tensor:
        return self.backbone(features)

    def forward(self, features: torch.Tensor) -> torch.Tensor:
        return self.classifier(self.embeddings(features))
```

Next, create a `SpectrogramCNN.yaml` configuration file for the model in the `conf/model/` directory:

conf/model/SpectrogramCNN.yaml

```yaml
id: SpectrogramCNN
_target_: spectrogram_cnn.SpectrogramCNN

hidden_dims: [32, 64, 128]

transform:
  type: grayscale
```

The `id` should match the name of the configuration file. The `_target_` should point to the custom model class via a python import path (here assuming that the `spectrogram_cnn.py` file is in the root directory of the project). Each model should include a `transform/type` attribute in the configuration file, specifying the input type it expects.

> **Note**
>
> The `output_dim` attribute is automatically passed to the model during initialization and determined by the dataset at runtime.
>
> The `transform` attribute in the configuration is not passed to the model during initialization and is used to specify the input type of the model and any online transforms to be applied to the data at runtime.

Custom Datasets

To create a custom dataset, inherit from `AbstractDataset` and implement the `target_transform` and `output_dim` properties.

The train, dev, and test datasets as well as loaders are automatically created by the abstract class. However, this requires that the dataset structure follows the standard format outlined in the dataset documentation. If the dataset structure is different or does not rely on dataframes, the `df_train`, `df_dev`, and `df_test`, `train_dataset`, `train_loader` etc. properties can be overridden.

autrainer provides base datasets for classification (`BaseClassificationDataset`), regression (`BaseRegressionDataset`), and multi-label classification (`BaseMLClassificationDataset`) tasks. In this case, both the target transform and output dimension are already implemented in the base class and do not need to be overridden.

> **Tip**
>
> To automatically download a custom dataset, implement the `download()` method. This method is called by the autrainer fetch CLI command as well as the `fetch()` CLI wrapper function. The `path` attribute specified in the dataset configuration file is passed to the method to store the downloaded data in.

ESC-50 Example

For example, the ESC-50 dataset is an audio classification dataset and can be implemented as follows:

esc_50.py

```python
from functools import cached_property
import os
import shutil
from typing import Any, Dict, List

import pandas as pd

from autrainer.datasets import BaseClassificationDataset
from autrainer.datasets.utils import ZipDownloadManager


FILES = {"ESC-50.zip": "https://github.com/karoldvl/ESC-50/archive/master.zip"}


class ESC50(BaseClassificationDataset):
    def __init__(
        self,
        train_folds: List[int],
        dev_folds: List[int],
        test_folds: List[int],
        **kwargs: Dict[str, Any],  # kwargs only for simplicity in the tutorial
    ) -> None:
        self.train_folds = train_folds
        self.dev_folds = dev_folds
        self.test_folds = test_folds
        super().__init__(**kwargs)

    @cached_property
    def _load_metadata(self) -> pd.DataFrame:
        return pd.read_csv(os.path.join(self.path, "esc50.csv"))

    @cached_property
    def df_train(self) -> pd.DataFrame:
        meta = self._load_metadata
        return meta[meta["fold"].isin(self.train_folds)]

    @cached_property
    def df_dev(self) -> pd.DataFrame:
        meta = self._load_metadata
        return meta[meta["fold"].isin(self.dev_folds)]

    @cached_property
    def df_test(self) -> pd.DataFrame:
        meta = self._load_metadata
        return meta[meta["fold"].isin(self.test_folds)]

    @staticmethod
    def download(path: str) -> None:
        if os.path.exists(os.path.join(path, "default")):
            return

        dl_manager = ZipDownloadManager(FILES, path)
        dl_manager.download(check_exist=["ESC-50.zip"])
        dl_manager.extract(check_exist=["ESC-50-master"])
        shutil.move(
            os.path.join(path, "ESC-50-master", "audio"),
            os.path.join(path, "default"),
        )
        shutil.move(
            os.path.join(path, "ESC-50-master", "meta", "esc50.csv"),
            path,
        )
        shutil.rmtree(os.path.join(path, "ESC-50-master"))
```

The dataset provides audio files by default (which are moved to the `default/` directory in the **`download()`** method) and the corresponding metadata of the dataset is stored in the `esc50.csv` file.

To allow the the specification of custom folds, the `df_train`, `df_dev`, and `df_test` properties are overridden to split the `esc50.csv` metadata file into the respective train, dev, and test dataframes. This also allows for cross-validation by creating multiple configurations with different folds.

To extract log-Mel spectrograms from the audio files, a preprocessing transform can be applied to the data before training. The following configuration creates a new `ESC50-32k.yaml` dataset in the `conf/dataset/` directory with log-Mel spectrograms preprocessed at a sample rate of 32 kHz:

conf/dataset/ESC50-32k.yaml

```yaml
id: ESC50-32k
_target_: esc_50.ESC50

path: data/ESC50
features_subdir: log_mel_32k
index_column: filename
target_column: category
file_type: npy
file_handler: autrainer.datasets.utils.NumpyFileHandler

train_folds: [1, 2, 3]
dev_folds: [4]
test_folds: [5]

criterion: autrainer.criterions.BalancedCrossEntropyLoss
metrics:
  - autrainer.metrics.Accuracy
  - autrainer.metrics.UAR
  - autrainer.metrics.F1
tracking_metric: autrainer.metrics.Accuracy

transform:
  type: grayscale
```

The dataset can be automatically downloaded and preprocessed using the autrainer fetch and autrainer preprocess CLI commands or the `fetch()` and `preprocess()` CLI wrapper functions.

Simple Dataset Example

If the structure of the dataset follows the standard format outlined in the dataset documentation, no implementation is necessary and a new dataset can be created by simply adding a configuration file to the `conf/dataset/` directory.

For example, the following configuration file creates a new `SpectrogramDataset.yaml` classification dataset, preprocessing the data with a spectrogram preprocessing transform at a sample rate of 32 kHz:

conf/dataset/SpectrogramDataset.yaml

```yaml
id: SpectrogramDataset-32k
_target_: autrainer.datasets.BaseClassificationDataset

path: data/SpectrogramDataset # base path to the dataset
features_subdir: log_mel_32k # spectrogram preprocessed features
index_column: path # column in the CSVs containing features paths relative to features_subdir
target_column: label # column in the CSVs containing the target labels
file_type: npy # file extension of the spectrogram features
file_handler: autrainer.datasets.utils.NumpyFileHandler # file handler for the spectrogram features

criterion: autrainer.criterions.BalancedCrossEntropyLoss
metrics:
  - autrainer.metrics.Accuracy
  - autrainer.metrics.UAR
  - autrainer.metrics.F1
tracking_metric: autrainer.metrics.Accuracy

transform:
  type: grayscale
```

This dataset assumes that the `data/SpectrogramDataset` directory contains the following directories and files:

- `default/` directory containing the raw audio files. These audio files are preprocessed using the spectrogram preprocessing transform with the autrainer preprocess CLI command or the `preprocess()` CLI wrapper function and stored in the `data/SpectrogramDataset/log_mel_32k` directory.
- `train.csv`, `dev.csv`, and `test.csv` files containing the file paths relative to the `default/` directory in the `index_column` column and the corresponding labels in the `target_column` column.

Custom Metrics

To create a custom metric, inherit from `AbstractMetric` and implement the `starting_metric`, `suffix` properties, as well as the `get_best()`, the `get_best_pos()`, and `compare()` static methods.

autrainer provides base classes for ascending (`BaseAscendingMetric`) and descending (`BaseDescendingMetric`) metrics that can be inherited from to simplify the implementation.

For example, the following metric implements the Cohen's Kappa score with either linear or quadratic weights:

cohens_kappa_metric.py

```python
import sklearn.metrics

from autrainer.metrics import BaseAscendingMetric


class CohensKappa(BaseAscendingMetric):
    def __init__(self, weights: str) -> None:
        """Coehn's Kappa metric using `sklearn.metrics.cohen_kappa_score`.

        Args:
            weights: Weighting type for the metric in ["linear", "quadratic"].
        """
        super().__init__(
            name="cohens-kappa",
            fn=sklearn.metrics.cohen_kappa_score,
            weights=weights,
        )
```

The `fn` attribute is the function that is automatically called in the `__call__()` method and the `weights` attribute is passed to the `fn` as a keyword argument.

As metrics are specified using shorthand syntax in the dataset configuration, the following relative import path can be used to reference it as the `tracking_metric` for the dataset:

conf/dataset/ExampleDataset.yaml

```yaml
...
tracking_metric:
  cohens_kappa_metric.CohensKappa:
    weights: linear # linear or quadratic
...
```

Custom Criterions

To create a custom criterion, inherit from `torch.nn.modules.loss._Loss` and implement the `forward()` method. If the criterion relies on the dataset, an optional criterion setup method can be defined which is called after the dataset is initialized.

> **Note**
>
> The `reduction` attribute of each criterion is automatically set to `"none"` during instantiation and the `forward()`

The `reduction` attribute of each criterion is automatically set to `none` during instantiation and the `forward()` method should return the per-example loss.

For example, the following criterion implements CrossEntropyLoss with an additional scaling factor:

scaled_ce_loss.py

```python
import torch


class ScaledCrossEntropyLoss(torch.nn.CrossEntropyLoss):
    def __init__(self, scaling_factor: float = 1.0, *args, **kwargs):
        """Cross entropy loss with a scaling factor.

        Args:
            scaling_factor: Scaling factor for the loss.
            *args: Positional arguments passed to `torch.nn.CrossEntropyLoss`.
            **kwargs: Keyword arguments passed to `torch.nn.CrossEntropyLoss`.
        """
        super().__init__(*args, **kwargs)
        self.scaling_factor = scaling_factor

    def forward(self, x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        if y.ndim == 1:
            y = y.long()
        return self.scaling_factor * super().forward(x, y)
```

As criterions are specified using shorthand syntax in the dataset configuration, the following relative import path can be used to reference it as the `criterion` for the dataset:

conf/dataset/ExampleDataset.yaml

```yaml
...
criterion:
  scaled_ce_loss.ScaledCrossEntropyLoss:
    scaling_factor: 0.5
...
```

Or without overriding the default `scaling_factor` value:

conf/dataset/ExampleDataset.yaml

```yaml
...
criterion: scaled_ce_loss.ScaledCrossEntropyLoss
...
```

Custom File Handlers

To create a custom file handler, inherit from `AbstractFileHandler` and implement the `load()` and `save()` methods.

For example, the following file handler loads and saves PyTorch tensors:

torch_file_handler.py

```python
import torch

from autrainer.datasets.utils import AbstractFileHandler


class TorchFileHandler(AbstractFileHandler):
    def load(self, file: str) -> torch.Tensor:
        return torch.load(file)

    def save(self, file: str, data: torch.Tensor) -> None:
```

```
        torch.save(data, file)
```

File handlers are specified using shorthand syntax in the dataset configuration. The following configuration utilizes the `TorchFileHandler` to load and save PyTorch tensors with the file extension `.pt`:

conf/dataset/ExampleDataset.yaml

```yaml
...
file_type: pt
file_handler: torch_file_handler.TorchFileHandler
...
```

Custom Target Transforms

To create a custom target transform, inherit from `AbstractTargetTransform` and implement the `encode()`, `decode()`, `predict_batch()`, and `majority_vote()` methods.

For example, the following target transform logarithmically encodes and decodes the targets for regression tasks:

log_target_transform.py

```python
import math
from typing import Dict, List, Union

import torch

from autrainer.datasets.utils import AbstractTargetTransform


class LogTargetTransform(AbstractTargetTransform):
    def __init__(self, target: str, base: int = 10, eps: float = 1e-9) -> None:
        """Logarithmic target transform for regression tasks.

        Args:
            target: Name of the target.
            base: Base of the logarithm. Defaults to 10.
            eps: Small value to avoid taking the logarithm of zero.
                Defaults to 1e-9.
        """
        self.target = target
        self.base = base
        self.eps = eps

    def encode(self, x: float) -> float:
        return math.log(x + self.eps, self.base)

    def decode(self, x: float) -> float:
        return math.pow(self.base, x) - self.eps

    def probabilities_inference(self, x: torch.Tensor) -> torch.Tensor:
        return x

    def predict_inference(self, x: torch.Tensor) -> Union[List[float], float]:
        return x.squeeze().tolist()

    def majority_vote(self, x: List[float]) -> float:
        return sum(x) / len(x)

    def probabilities_to_dict(self, x: torch.Tensor) -> Dict[str, float]:
        return {self.target: x.item()}
```

The target transforms are specified in the `target_transform` property of a dataset implementation.

Advanced Data Pipelines

To create data and model pipelines that go beyond the standard `DataItem` convention of using only `features` as input to the

model, first create a new `DataItem` dataclass that includes a new parameter (e.g., `meta`):

multi_branch_data.py

```
@dataclass
class DataItemMultiBranch(AbstractDataItem):
    features: torch.Tensor
    meta: torch.Tensor
    target: int
    index: int
```

Next, override `AbstractDataBatch` to include the new `meta` parameter:

multi_branch_data.py

```
@dataclass
class DataBatchMulti(AbstractDataBatch[DataItemMultiBranch]):
    """Data batch class for a batch of data samples.

    Args:
        features: Tensor of input features.
        meta: Tensor of input support features.
        target: Tensor of target values for the input features.
        index: Tensor of indices for the data samples.
    """

    features: torch.Tensor
    meta: torch.Tensor
    target: torch.Tensor
    index: torch.Tensor

    def to(self, device: torch.device) -> None:
        self.features = self.features.to(device)
        self.meta = self.meta.to(device)
        self.target = self.target.to(device)
```

Following that, inherit from `DatasetWrapper` to create a `torch.utils.data.Dataset` that iterates over your data and returns your custom `AbstractDataBatch` (here we simply replicate `features` as our auxiliary features):

multi_branch_data.py

```
class ToyDatasetMultiBranch(ToyDatasetWrapper):
    def __getitem__(self, index: int) -> DataItemMultiBranch:
        data = super().__getitem__(index)
        return DataItemMultiBranch(
            features=data.features,
            target=data.target,
            index=data.index,
            meta=data.features,
        )
```

Subsequently, inherit from `AbstractDataset` to create a dataset that instantiates your `DatasetWrapper`:

multi_branch_data.py

```
class ToyMultiBranchData(ToyDataset):
    def _init_dataset(
        self,
        df: pd.DataFrame,
        transform: SmartCompose,
    ) -> ToyDatasetMultiBranch:
        return ToyDatasetMultiBranch(
            df=df,
            target_column=self.target_column,
            feature_shape=self.feature_shape,
            dtype=self.dtype,
            generator=self._generator,
            transform=transform,
```

```
            target_transform=self.target_transform,
        )

    @property
    def default_collate_fn(self):
        return DataBatchMulti.collate
```

Next, create a `ToyMultiBranch-C.yaml` configuration file for the dataset in the `conf/dataset/` directory:

conf/data/ToyMultiBranch-C.yaml

```yaml
id: ToyMultiBranch-C
_target_: multi_branch_data.ToyMultiBranchData

task: classification
size: 1000
num_targets: 10
feature_shape: 64
dev_split: 0.2
test_split: 0.2

criterion: autrainer.criterions.BalancedCrossEntropyLoss
metrics:
  - autrainer.metrics.Accuracy
  - autrainer.metrics.UAR
  - autrainer.metrics.F1
tracking_metric: autrainer.metrics.Accuracy

transform:
  type: tabular
```

Finally, inherit from `AbstractModel` and create a model **with a matching signature**, in its forward pass to access the `meta` parameter:

multi_branch_model.py

```python
class ToyMultiBranchModel(AbstractModel):
    def __init__(self, input_dim: int, output_dim: int, hidden_dim: int):
        super().__init__(output_dim, None)  # no transfer learning
        self.input_dim = input_dim
        self.hidden_dim = hidden_dim

        self.linear1 = torch.nn.Linear(self.input_dim, self.hidden_dim)
        self.linear2 = torch.nn.Linear(self.input_dim, self.hidden_dim)
        self.out = torch.nn.Linear(self.hidden_dim * 2, self.output_dim)

    def embeddings(
        self, features: torch.Tensor, meta: torch.Tensor
    ) -> torch.Tensor:
        return torch.concat(
            [self.linear1(features), self.linear2(meta)], axis=1
        )

    def forward(
        self, features: torch.Tensor, meta: torch.Tensor
    ) -> torch.Tensor:
        return self.out(self.embeddings(features=features, meta=meta))
```

Next, create a `ToyMultiBranchModel.yaml` configuration file for the model in the `conf/model/` directory:

conf/data/ToyMultiBranchModel.yaml

```yaml
id: ToyMultiBranchModel
_target_: multi_branch_model.ToyMultiBranchModel
input_dim: 64
hidden_dim: 64

transform:
  type: tabular
```

Custom Optimizers

To create a custom optimizer, inherit from torch.optim.Optimizer and implement the `step()` method.

For example, the following optimizer implements the SGD optimizer with an additional randomly scaled learning rate using a custom step function:

random_scaled_sgd.py

```python
from typing import Callable, Tuple

import torch

from autrainer.core.structs import AbstractDataBatch
from autrainer.models import AbstractModel
from autrainer.models.utils import create_model_inputs


class RandomScaledSGD(torch.optim.Optimizer):
    def __init__(
        self,
        scaling_factor: float = 0.01,
        p: float = 1.0,
        generator_seed: int = None,
        *args,
        **kwargs,
    ) -> None:
        """Randomized Scaled SGD optimizer. Randomly scales the learning rate.

        Args:
            scaling_factor: Learning rate scaling factor. Defaults to 1.0.
            p: Probability of scaling the learning rate. Defaults to 1.0.
            generator_seed: Seed for the random number generator.
                Defaults to None.
        """
        super().__init__(*args, **kwargs)
        self.scaling_factor = scaling_factor
        self.p = p
        self.g = torch.Generator()
        self.base_lr = self.param_groups[0]["lr"]
        if generator_seed is not None:
            self.g.manual_seed(generator_seed)

    def custom_step(
        self,
        model: AbstractModel,  # model
        data: AbstractDataBatch,  # batched input data
        criterion: torch.nn.Module,  # loss function
        probabilities_fn: Callable,  # function to get probabilities from model outputs
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        self.zero_grad()
        output = model(**create_model_inputs(model, data))
        loss = criterion(probabilities_fn(output), data.target)
        loss.mean().backward()
        if torch.rand(1, generator=self.g).item() < self.p:
            self.param_groups[0]["lr"] *= self.scaling_factor
        self.step()
        self.param_groups[0]["lr"] = self.base_lr
        return loss, output
```

The following configuration creates a new `RandomScaledSGD.yaml` optimizer in the `conf/optimizer/` directory and uses the global seed of the main configuration as the `generator_seed` attribute:

conf/optimizer/RandomScaledSGD.yaml

```yaml
id: RandomScaledSGD
_target_: random_scaled_sgd.RandomScaledSGD

scaling_factor: 0.001
```

```yaml
p: 0.05
generator_seed: ${seed}
```

> **Note**
>
> The `params` and `lr` attributes are automatically passed to the optimizer during initialization and determined at runtime.

Custom Schedulers

To create a custom scheduler, inherit from torch.optim.lr_scheduler.LRScheduler and implement the `get_lr()` method.

For example, the following scheduler implements a simple linear warm-up scheduler:

linear_warm_up_lr.py

```python
from typing import List

import torch
from torch.optim.lr_scheduler import LRScheduler


class LinearWarmUpLR(LRScheduler):
    def __init__(
        self,
        optimizer: torch.optim.Optimizer,
        warmup_steps: int,
        last_epoch: int = -1,
    ) -> None:
        """Linear warm-up learning rate scheduler.

        Args:
            optimizer: Wrapped optimizer.
            warmup_steps: Number of warmup steps.
            last_epoch: The index of last epoch. Defaults to -1.
        """
        self.warmup_steps = warmup_steps
        super().__init__(optimizer, last_epoch)

    def get_lr(self) -> List[float]:
        if self.last_epoch < self.warmup_steps:
            return [
                base_lr * (self.last_epoch + 1) / self.warmup_steps
                for base_lr in self.base_lrs
            ]
        return self.base_lrs
```

The following configuration creates a new `LinearWarmUpLR.yaml` scheduler with a linear warm-up period of 10 training iterations in the `conf/scheduler/` directory:

conf/scheduler/LinearWarmUpLR.yaml

```yaml
id: LinearWarmUpLR
_target_: linear_warm_up_lr.LinearWarmUpLR

warmup_steps: 10

step_frequency: evaluation
```

> **Note**
>
> The `optimizer` attribute is automatically passed to the scheduler during initialization and determined at runtime.

Custom Transforms

To create a custom transform, inherit from `AbstractTransform` and implement the `__call__()` method.

For example, the following transform denoises a spectrogram by applying a median filter:

spect_median_filter.py

```python
import scipy.ndimage
import torch

from autrainer.core.structs import AbstractDataItem
from autrainer.transforms import AbstractTransform


class SpectMedianFilter(AbstractTransform):
    def __init__(self, size: int, order: int = 0) -> None:
        """Spectrogram median filter to remove noise.

        Args:
            size: Number of neighboring pixels to consider when filtering.
                Must be odd.
            order: The order of the transform in the pipeline. Larger means
                later in the pipeline. If multiple transforms have the same
                order, they are applied in the order they were added to the
                pipeline. Defaults to 0.
        """
        super().__init__(order=order)
        self.size = size

    def __call__(self, item: AbstractDataItem) -> AbstractDataItem:
        item.features = torch.from_numpy(
            scipy.ndimage.median_filter(
                item.features.cpu().numpy(),
                size=self.size,
            )
        ).to(item.features.device)
        return item
```

This transform can be used both as a preprocessing transform and as an online transform.

Custom Preprocessing Transforms

To create a custom preprocessing transform, create a new file in the `conf/preprocessing/` directory.

For example, the following preprocessing transform extracts log-Mel spectrograms from audio data at a sampling rate of 32 kHz and applies the custom denoising transform to the data:

conf/scheduler/denoised_log_mel_32k.yaml

```yaml
file_handler:
  autrainer.datasets.utils.AudioFileHandler:
    target_sample_rate: 32000
pipeline:
  - autrainer.transforms.StereoToMono
  - autrainer.transforms.PannMel:
      sample_rate: 32000
      window_size: 1024
      hop_size: 320
      mel_bins: 64
      fmin: 50
      fmax: 14000
      ref: 1.0
      amin: 1e-10
      top_db: null
  - spect_median_filter.SpectMedianFilter:
      size: 5
```

Any audio dataset can utilize this preprocessing transform by specifying the `features_subdir` attribute in the dataset configuration and adjusting the `file_type`, `file_handler`, and `transform` attributes:

conf/dataset/ExampleDataset.yaml

```yaml
...
features_subdir: denoised_log_mel_32k
file_type: npy
file_handler: autrainer.datasets.utils.NumpyFileHandler
...
transform:
  type: grayscale
```

> **Note**
>
> The `save()` method of the `file_handler` specified in the dataset configuration is used to save the processed data to the `features_subdir` directory. The `load()` method of the `file_handler` is used to load the processed data during training and inference.

Custom Online Transforms

To create a custom online transform, no configuration file is necessary as the transform is applied at runtime and specified in the `transform` attribute of the model and dataset configurations using shorthand syntax.

For example, the following configuration applies the custom denoising transform to the data at runtime:

conf/dataset/ExampleDataset.yaml

```yaml
...
transform:
  type: grayscale
  base:
    - spect_median_filter.SpectMedianFilter:
        size: 5
```

In line with the custom preprocessing transform example, the custom denoising transform is applied to the train, dev, and test datasets.

It may be desirable to only apply a transform to a specific subset of the data. The following configuration applies the custom denoising transform only to the `train` subset of the data:

conf/dataset/ExampleDataset.yaml

```yaml
...
transform:
  type: grayscale
  train:
    - spect_median_filter.SpectMedianFilter:
        size: 5
```

Custom Augmentations

To create a custom augmentation, inherit from `AbstractAugmentation` and implement the `apply()` method.

For example, the following augmentation scales the amplitude of a spectrogram by a random factor in a given range:

amplitude_scale_augmentation.py

```python
from typing import Optional, Tuple

import torch

from autrainer.augmentations import AbstractAugmentation
from autrainer.core.structs import AbstractDataItem


class AmplitudeScale(AbstractAugmentation):
    def __init__(
        self,
        scale_range: Tuple[float, float],
        order: int = 0,
        p: float = 1.0,
        generator_seed: Optional[int] = None,
    ) -> None:
        """Amplitude scaling augmentation. The amplitude is randomly scaled by
        a factor drawn from scale_range.

        Args:
            scale_range: The range of the amplitude scaling factor.
            order: The order of the augmentation in the transformation pipeline.
                Defaults to 0.
            p: The probability of applying the augmentation. Defaults to 1.0.
            generator_seed: The initial seed for the internal random number
                generator drawing the probability. If None, the generator is
                not seeded. Defaults to None.

        Raises:
            ValueError: If p is not in the range [0, 1].
        """
        super().__init__(order, p, generator_seed)
        self.scale_range = scale_range
        self.scale_g = torch.Generator()
        if self.generator_seed is not None:
            self.scale_g.manual_seed(self.generator_seed)

    def apply(self, item: AbstractDataItem) -> AbstractDataItem:
        s0, s1 = self.scale_range
        scale = torch.rand(1, generator=self.scale_g)
        item.features = item.features * (scale * (s1 - s0) + s0)
        return item
```

The following configuration creates a new `AmplitudeScale.yaml` augmentation in the `conf/augmentation/` directory, scaling the amplitude of the spectrogram by a random factor between 0.8 and 1.2 with a probability `p` of 0.5:

conf/augmentation/AmplitudeScale.yaml

```yaml
id: AmplitudeScale
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - amplitude_scale_augmentation.AmplitudeScale:
      scale_range: [0.8, 1.2]
      p: 0.5
```

As no augmentation in the `pipeline` specifies a `generator_seed` attribute, the global `generator_seed` attribute is broadcasted to all augmentations to ensure reproducibility.

Custom Augmentation Graphs

For example, the following configuration creates a new `AmplitudeScaleOrTimeFreqMask.yaml` augmentation in the `conf/augmentation/` directory, either applying the custom amplitude scale augmentation or a sequence of the `TimeMask` and `FrequencyMask` augmentations:

conf/augmentation/AmplitudeScaleOrTimeFreqMask.yaml

```yaml
id: AmplitudeScaleOrTimeFreqMask
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.Choice:
      weights: [0.2, 0.8]
      choices:
        - amplitude_scale_augmentation.AmplitudeScale:
            scale_range: [0.8, 1.2]
        - autrainer.augmentations.Sequential:
            sequence:
              - autrainer.augmentations.TimeMask:
                  time_mask: 80
              - autrainer.augmentations.FrequencyMask:
                  freq_mask: 10
```

The custom amplitude scale augmentation is selected with a probability of 0.2, while the sequence of the `TimeMask` and `FrequencyMask` augmentations is selected with a probability of 0.8.

Custom Collate Augmentations

To create a custom collate augmentation, inherit from `AbstractAugmentation` and implement the optional `get_collate_fn()` method.

The collate function is used to apply the augmentation on the batch level. In case the collate function modifies the shape of the input or labels, this may need to be accounted for if the augmentation is not applied.

For example, the following augmentation randomly applies `CutMix` or `MixUp` augmentations on the batch level:

cut_mix_up.py

```python
from typing import TYPE_CHECKING, Callable, List, Optional

import torch
from torchvision.transforms import v2

from autrainer.augmentations import AbstractAugmentation
from autrainer.core.structs import AbstractDataBatch, AbstractDataItem


if TYPE_CHECKING:
    from autrainer.datasets import AbstractDataset


class CutMixUp(AbstractAugmentation):
    def __init__(
        self,
        alpha: float = 1.0,
        order: int = 0,
        p: float = 1.0,
        generator_seed: Optional[int] = None,
    ) -> None:
        """Randomly applies CutMix or MixUp augmentations with a probability
        of 0.5 each.

        Args:
            alpha: Hyperparameter of the Beta distribution. Defaults to 1.0.
            order: The order of the augmentation in the transformation pipeline.
                Defaults to 0.
            p: The probability of applying the augmentation. Defaults to 1.0.
            generator_seed: The initial seed for the internal random number
                generator drawing the probability. If None, the generator is
                not seeded. Defaults to None.
        """
        super().__init__(order, p, generator_seed)
        self.alpha = alpha
        self.cut_mix_up_g = torch.Generator()
        if generator_seed is not None:
```

```
            self.cut_mix_up_g.manual_seed(generator_seed)

    def get_collate_fn(
        self,
        data: "AbstractDataset",
        default: Callable,
    ) -> Callable:
        self.cutmix = v2.CutMix(num_classes=data.output_dim, alpha=self.alpha)
        self.mixup = v2.MixUp(num_classes=data.output_dim, alpha=self.alpha)

        def _collate_fn(batch: List[AbstractDataItem]) -> AbstractDataBatch:
            probability = torch.rand(1, generator=self.g).item()
            batched: AbstractDataBatch = default(batch)
            if probability < self.p:
                features = batched.features
                target = batched.target
                if probability < 0.5:
                    results = self.cutmix(features, target)
                else:
                    results = self.mixup(features, target)
                batched.features = results[0]
                batched.target = results[1]
                return batched
            batched.target = torch.nn.functional.one_hot(
                batched.target, data.output_dim
            ).float()
            return batched

        return _collate_fn

    def apply(self, item: AbstractDataItem) -> AbstractDataItem:
        # no-op as the augmentation is applied in the collate function
        return item
```

Custom Loggers

To create a custom logger, inherit from `AbstractLogger` and implement the `log_params()`, `log_metrics()`, `log_timers()`, and `log_artifact()` methods, as well as the optional `setup()`, and `end_run()` methods.

All methods are automatically called a the appropriate time during training and inference.

For example, the following logger logs to Weights & Biases:

wandb_logger.py

```
import os
from typing import Dict, List, Union

from omegaconf import DictConfig
import wandb

from autrainer.core.constants import ExportConstants
from autrainer.loggers import (
    AbstractLogger,
    get_params_to_export,
)
from autrainer.metrics import AbstractMetric


class WandBLogger(AbstractLogger):
    def __init__(
        self,
        exp_name: str,
        run_name: str,
        metrics: List[AbstractMetric],
        tracking_metric: AbstractMetric,
        artifacts: List[
            Union[str, Dict[str, str]]
        ] = ExportConstants().ARTIFACTS,
        output_dir: str = "wandb",
    ) -> None:
        super().__init__(
            exp_name, run_name, metrics, tracking_metric, artifacts
```

```
            exp_name, run_name, metrics, tracking_metric, artifacts
        )
        if not os.path.isabs(output_dir):
            output_dir = os.path.join(os.getcwd(), output_dir)
        os.makedirs(output_dir, exist_ok=True)
        self.output_dir = output_dir

    def log_params(self, params: Union[dict, DictConfig]) -> None:
        wandb.init(
            project=self.exp_name,
            name=self.run_name,
            config=get_params_to_export(params),
            dir=self.output_dir,
        )

    def log_metrics(
        self,
        metrics: Dict[str, Union[int, float]],
        iteration=None,
    ) -> None:
        wandb.log(metrics, step=iteration)

    def log_timers(self, timers: Dict[str, float]) -> None:
        wandb.log(timers)

    def log_artifact(self, filename: str, path: str = "") -> None:
        artifact = wandb.Artifact(name=filename, type="model")
        artifact.add_file(os.path.join(path, filename))
        wandb.log_artifact(artifact)

    def end_run(self) -> None:
        wandb.finish()
```

Note that the `WandBLogger` assumes that wandb is installed, the API key is set, and a project with the same name as the `experiment_id` of the main configuration exists.

To add the `WandBLogger`, specify it in the main configuration by adding a list of `loggers`:

conf/config.yaml

```
...
loggers:
  - wandb_logger.WandBLogger:
      output_dir: ${results_dir}/.wandb
...
```

Custom Callbacks

To create a custom callback, implement a class that specifies any of the callback functions defined in `CallbackSignature`.

For example, the following callback tracks learning rate changes at the beginning of each iteration:

lr_tracker_callback.py

```
from typing import TYPE_CHECKING


if TYPE_CHECKING:
    from autrainer.training import ModularTaskTrainer


class LRTrackerCallback:
    def cb_on_train_begin(self, trainer: "ModularTaskTrainer") -> None:
        self.lr = trainer.optimizer.param_groups[0]["lr"]

    def cb_on_iteration_begin(
        self,
        trainer: "ModularTaskTrainer",
        iteration: int,
    ) -> None:
```

```
        current_lr = trainer.optimizer.param_groups[0]["lr"]
        if current_lr != self.lr:
            print(
                f"Learning rate changed from {self.lr} "
                f"to {current_lr} in iteration {iteration}."
            )
            self.lr = current_lr
```

To add the `LRTrackerCallback`, specify it in the main configuration by adding a list of `callbacks`:

conf/config.yaml

```
...
callbacks:
  - lr_tracker_callback.LRTrackerCallback
...
```

Custom Plotting

To create a custom plotting configuration, create a new file in the `conf/plotting/` directory.

For example, the following configuration uses the LaTeX backend, the Palatino font with a font size of 9, replaces `None` values in the run name with `~` for better readability, and adds labels as well as titles to the plot.

conf/plotting/LaTeX.yaml

```
figsize: [10, 5] # figure size in inches
latex: true # use LaTeX for text rendering
filetypes: [png, pdf] # save figures in these formats
pickle: true # save the figure data in a pickle file
context: notebook # seaborn context
palette: colorblind # seaborn color palette
replace_none: true # replace None with ~
add_titles: true
add_xlabels: true
add_ylabels: true

rcParams:
  font.serif: Palatino # LaTeX font
  font.family: serif
  legend.fontsize: 9
```

To add the `LaTeX.yaml` plotting configuration, specify it in the main configuration by overriding the `plotting` attribute:

conf/config.yaml

```
defaults:
  - ...
  - override plotting: LaTeX
...
```

CLI Reference

autrainer provides a command line interface (CLI) to manage the entire training process, including configuration management, data preprocessing, model training, inference, and postprocessing.

In addition to the CLI, *autrainer* provides an CLI wrapper to manage configurations, data, training, inference, and postprocessing programmatically with the same functionality as the CLI.

autrainer

A Modular and Extensible Deep Learning Toolkit for Computer Audition Tasks.

```
usage: autrainer [-h] [-v] <command> ...
```

Positional Arguments

`<command>`

Possible choices: create, list, show, fetch, preprocess, train, inference, postprocess, rm-failed, rm-states, group

Named Arguments

`-v, --version`

show program's version number and exit

Configuration Management

To manage configurations, autrainer create, autrainer list, and autrainer show allow for the creation of the project structure and the discovery as well as saving of default configurations provided by *autrainer*.

> **Tip**
>
> Default configurations can be discovered both through the CLI, the CLI wrapper, and the respective module documentation.

autrainer create

Create a new project with default configurations.

```
usage: autrainer create [-h] [-e] [-a] [-f] [directories ...]
```

Positional Arguments

`directories`

Configuration directories to create. One or more of:

- augmentation
- dataset
- model
- optimizer
- plotting
- preprocessing
- scheduler

Named Arguments

`-e, --empty`

Create an empty project without any configuration directory.

`-a, --all`

Create a project with all configuration directories.

`-f, --force`

Force overwrite if the configuration directory already exists.

autrainer list

List local and global configurations.

```
usage: autrainer list [-h] [-l] [-g] [-p P] directory
```

Positional Arguments

`directory`

The directory to list configurations from. Choose from:

- augmentation
- dataset
- model
- optimizer
- plotting
- preprocessing
- scheduler

Named Arguments

`-l, --local-only`

List local configurations only.

`-g, --global-only`

List global configurations only.

`-p, --pattern`

Glob pattern to filter configurations.

autrainer show

Show and save a global configuration.

```
usage: autrainer show [-h] [-s] [-f] directory config
```

Positional Arguments

`directory`

The directory to list configurations from. Choose from:

- augmentation
- dataset
- model
- optimizer
- plotting

- plotting
- preprocessing
- scheduler

`config`

The global configuration to show. Configurations can be discovered using the 'autrainer list' command.

Named Arguments

`-s, --save`

Save the global configuration to the local conf/ directory.

`-f, --force`

Force overwrite local configuration if it exists in combination with -s/–save.

Preprocessing

To avoid race conditions when using Launcher Plugins that may run multiple training jobs in parallel, autrainer fetch and autrainer preprocess allow for downloading and preprocessing of Datasets (and pretrained model states or transforms) before training.

Both commands are based on the main configuration file (e.g., `conf/config.yaml`), such that the specified models and datasets are fetched and preprocessed accordingly. If a model or dataset is already fetched or preprocessed, it will be skipped.

autrainer fetch

Fetch the datasets, models, and transforms specified in a training configuration (Hydra). For more information on Hydra's command line line flags, see: https://hydra.cc/docs/advanced/hydra-command-line-flags/.

```
usage: autrainer fetch [-h] [-b]
```

Named Arguments

`-l, --cfg-launcher`

Use the launcher specified in the configuration instead of the Hydra basic launcher. Defaults to False.

autrainer preprocess

Launch a data preprocessing configuration (Hydra). For more information on Hydra's command line line flags, see: https://hydra.cc/docs/advanced/hydra-command-line-flags/.

```
usage: autrainer preprocess [-h] [-b] [-n N] [-p P] [-s]
```

Named Arguments

`-l, --cfg-launcher`

Use the launcher specified in the configuration instead of the Hydra basic launcher. Defaults to False.

`-n, --num-workers`

Number of workers (subprocesses) to use for preprocessing. Defaults to 0.

`-u, --update-frequency`

`-u, --update-frequency`

Frequency of progress bar updates for each worker (subprocess). If 0, the progress bar will be disabled. Defaults to 1.

Training

Training is managed by autrainer train, which starts the training process based on the main configuration file (e.g., `conf/config.yaml`).

autrainer train

Launch a training configuration (Hydra). For more information on Hydra's command line line flags, see: https://hydra.cc/docs/advanced/hydra-command-line-flags/.

```
usage: autrainer train [-h]
```

Inference

autrainer inference allows for the (sliding window) inference of audio data using a trained model.

Both local paths and Hugging Face Hub links are supported for the model. Hugging Face Hub links are automatically downloaded and cached in the torch cache directory.

The following syntax is supported for Hugging Face Hub links: `hf:repo_id[@revision][:subdir]#local_dir`. This syntax consists of the following components:

- `hf`: The Hugging Face Hub prefix indicating that the model is fetched from the Hugging Face Hub.
- `repo_id`: The repository ID of the model consisting of the user name and the model card name separated by a slash (e.g., `autrainer/example`).
- `revision` (*optional*): The revision as a commit hash, branch name, or tag name (e.g., `main`). If not specified, the latest revision is used.
- `subdir` (*optional*): The subdirectory of the repository containing the model directory (e.g., `AudioModel`). If not specified, the model directory is automatically inferred. If multiple models are present in the `repo_id`, `subdir` must be specified, as the correct model cannot be automatically inferred.
- `local_dir` (*optional*): The local directory to which the model is downloaded to (e.g., `.hf_local`). If not specified, the model is placed in the torch hub cache directory.

For example, to download the model from the repository `autrainer/example` at the revision `main` from the subdirectory `AudioModel` and save it to the local directory `.hf_local`, the following autrainer inference CLI command can be used:

```
autrainer inference hf:autrainer/example@main:AudioModel#.hf_local input/ output/ -d cuda:0
```

> **Tip**
>
> To access private repositories, the environment variable `HF_HOME` should point to the Hugging Face User Access Token.
>
> To use a custom endpoint (e.g., for a self-hosted hub), the environment variable `HF_ENDPOINT` should point to the desired endpoint URL.

To use a local model path, the following autrainer inference CLI command can be used:

```
autrainer inference /path/to/AudioModel input/ output/ -d cuda:0
```

autrainer inference

Perform inference on a trained model.

```
usage: autrainer inference [-h] [-c C] [-d D] [-e E] [-r] [-emb] [-p P] [-w W] [-s S] [-m M] [-sr SR] model
```

Positional Arguments

`model`

Local path to model directory or Hugging Face link of the format: *hf:repo_id[@revision][:subdir]#local_dir*. Should contain at least one state subdirectory, the *model.yaml*, *file_handler.yaml*, *target_transform.yaml*, and *inference_transform.yaml* files.

`input`

Path to input directory. Should contain audio files of the specified extension.

`output`

Path to output directory. Output includes a YAML file with predictions and a CSV file with model outputs.

Named Arguments

`-c, --checkpoint`

Checkpoint to use for evaluation. Defaults to '_best' (on validation set).

`-d, --device`

CUDA-enabled device to use for processing. Defaults to 'cpu'.

`-e, --extension`

Type of file to look for in the input directory. Defaults to 'wav'.

`-r, --recursive`

Recursively search for files in the input directory. Defaults to False.

`-emb, --embeddings`

Extract embeddings from the model in addition to predictions.For each file, a .pt file with embeddings will be saved.Defaults to False.

`-u, --update-frequency`

Frequency of progress bar updates. If 0, the progress bar will be disabled. Defaults to 1.

`-p, --preprocess-cfg`

Preprocessing configuration to apply to input. Can be a path to a YAML file or the name of the preprocessing configuration in the local or autrainer 'conf/preprocessing' directory. If 'default', the default preprocessing configuration used during training will be applied. If 'None', no preprocessing will be applied. Defaults to 'default'.

`-w, --window-length`

Window length for sliding window inference in seconds. If None, the entire input will be processed at once. Defaults to None.

`-s, --stride-length`

Stride length for sliding window inference in seconds. If None, the entire input will be processed at once. Defaults to None.

`-m, --min-length`

Minimum length of audio file to process in seconds. Files shorter than the minimum length are padded with zeros. Sample rate has to be specified for padding. If None, no minimum length is enforced. Defaults to None.

`-sr, --sample-rate`

Sample rate of audio files in Hz. Has to be specified for sliding window inference. Defaults to None.

`-t, --trust-remote`

Whether to trust and execute custom Python code from Hugging Face models. Warning: This can be dangerous as it may run arbitrary code on your machine! Defaults to False.

Postprocessing

Postprocessing allows for the summarization, visualization, and aggregation of the training results using autrainer postprocess. Several cleanup utilities are provided by autrainer rm-failed and autrainer rm-states. Manual grouping of the training results can be done using autrainer group.

autrainer postprocess

Postprocess grid search results.

```
usage: autrainer postprocess [-h] [-m N] [-a A [A ...]] results_dir experiment_id
```

Positional Arguments

`results_dir`

Path to grid search results directory.

`experiment_id`

ID of experiment to postprocess.

Named Arguments

`-m, --max-runs`

Maximum number of best runs to plot.

`-a, --aggregate`

Configurations to aggregate. One or more of:

- augmentation
- batch_size
- dataset
- iterations
- learning_rate
- model
- optimizer
- scheduler
- seed

autrainer rm-failed

Delete failed runs from an experiment.

```
usage: autrainer rm-failed [-h] [-f] results_dir experiment_id
```

Positional Arguments

`results_dir`

Path to grid search results directory.

`experiment_id`

ID of experiment to postprocess.

Named Arguments

`-f, --force`

Force deletion of failed runs without confirmation. Defaults to False.

autrainer rm-states

Delete states (.pt files) from an experiment.

```
usage: autrainer rm-states [-h] [-b] [-r R [R ...]] [-i I [I ...]] results_dir experiment_id
```

Positional Arguments

`results_dir`

Path to grid search results directory.

`experiment_id`

ID of experiment to postprocess.

Named Arguments

`-b, --keep-best`

Keep best states. Defaults to True.

`-r, --keep-runs`

Runs to keep.

`-i, --keep-iterations`

Iterations to keep.

autrainer group

Launch a manual grouping of multiple grid search results (Hydra). For more information on Hydra's command line line flags, see: https://hydra.cc/docs/advanced/hydra-command-line-flags/.

```
usage: autrainer group [-h]
```

CLI Wrapper

CLI Wrapper

autrainer provides an `autrainer.cli` CLI wrapper to programmatically manage the entire training process, including configuration management, data preprocessing, model training, inference, and postprocessing.

Wrapper functions are useful for integrating *autrainer* into custom scripts, jupyter notebooks, google colab notebooks, and other applications.

In addition to the CLI wrapper functions, *autrainer* provides a CLI to manage configurations, data, training, inference, and postprocessing from the command line with the same functionality as the CLI wrapper.

Configuration Management

To manage configurations, `create()`, `list()`, and `show()` allow for the creation of the project structure and the discovery as well as saving of default configurations provided by *autrainer*.

> **Tip**
>
> Default configurations can be discovered both through the CLI, the CLI wrapper, and the respective module documentation.

autrainer.cli.create(*directories=None, empty=False, all=False, force=False*)

Create a new project with default configurations.

If called in a notebook, the function will not raise an error and print the error message instead.

Parameters:

- **directories** (`Optional[List[str]]`) – Configuration directories to create. One or more of: `CONFIG_DIRS`. Defaults to None.
- **empty** (`bool`) – Create an empty project without any configuration directory. Defaults to False.
- **all** (`bool`) – Create a project with all configuration directories. Defaults to False.
- **force** (`bool`) – Force overwrite if the configuration directory already exists. Defaults to False.

Raises:

- **CommandLineError** – If no configuration directories are specified and neither the empty nor all flags are set.
- **CommandLineError** – If the empty and all flags are set at the same time.
- **CommandLineError** – If the empty or all flags are set in combination with configuration directories.
- **CommandLineError** – If the configuration directory already exists and the force flag is not set.

Return type:

`None`

autrainer.cli.list(*directory, local_only=False, global_only=False, pattern='*'*)

List local and global configurations.

If called in a notebook, the function will not raise an error and print the error message instead.

Parameters:

- **directory** (`str`) – The directory to list configurations from. Choose from: `CONFIG_DIRS`.
- **local_only** (`bool`) – List local configurations only. Defaults to False.
- **global_only** (`bool`) – List global configurations only. Defaults to False.
- **pattern** (`str`) – Glob pattern to filter configurations. Defaults to "*".

Raises:

Raises:

CommandLineError – If the local configuration directory does not exist and local_only is True.

Return type:

`None`

autrainer.cli.show(*directory*, *config*, *save=False*, *force=False*)

Show and save a global configuration.

If called in a notebook, the function will not raise an error and print the error message instead.

Parameters:

- **directory** (`str`) – The directory to list configurations from. Choose from: `CONFIG_DIRS`.
- **config** (`str`) – The global configuration to show. Configurations can be discovered using the 'autrainer list' command.
- **save** (`bool`) – Save the global configuration to the local conf directory. Defaults to False.
- **force** (`bool`) – Force overwrite local configuration if it exists in combination with save=True. Defaults to False.

Raises:

- **CommandLineError** – If the global configuration does not exist.
- **CommandLineError** – If while saving the local configuration, the configuration already exists and force is not set.

Return type:

`None`

Preprocessing

To avoid race conditions when using Launcher Plugins that may run multiple training jobs in parallel, `fetch()` and `preprocess()` allow for downloading and preprocessing of Datasets (and pretrained model states or transforms) before training.

Both commands are based on the main configuration file (e.g., `conf/config.yaml`), such that the specified models and datasets are fetched and preprocessed accordingly. If a model or dataset is already fetched or preprocessed, it will be skipped.

autrainer.cli.fetch(*override_kwargs=None*, *cfg_launcher=False*, *config_name='config'*, *config_path=None*)

Fetch the datasets, models, and transforms specified in a training configuration.

Parameters:

- **override_kwargs** (`Optional[dict]`) – Additional Hydra override arguments to pass to the train script.
- **cfg_launcher** (`bool`) – Use the launcher specified in the configuration instead of the Hydra basic launcher. Defaults to False.
- **config_name** (`str`) – The name of the config (usually the file name without the .yaml extension). Defaults to "config".
- **config_path** (`Optional[str]`) – The config path, a directory where Hydra will search for config files. If config_path is None no directory is added to the search path. Defaults to None.

Return type:

`None`

autrainer.cli.preprocess(*override_kwargs=None*, *num_workers=0*, *update_frequency=1*, *cfg_launcher=False*, *config_name='config'*, *config_path=None*)

Launch a data preprocessing configuration

Launch a data preprocessing configuration.

Parameters:

- **override_kwargs** (`Optional[dict]`) – Additional Hydra override arguments to pass to the train script.
- **num_workers** (`int`) – Number of workers (subprocess) to use for preprocessing. Defaults to 0.
- **update_frequency** (`int`) – Frequency of progress bar updates. If 0, the progress bar will be disabled. Defaults to 1.
- **cfg_launcher** (`bool`) – Use the launcher specified in the configuration instead of the Hydra basic launcher. Defaults to False.
- **config_name** (`str`) – The name of the config (usually the file name without the .yaml extension). Defaults to "config".
- **config_path** (`Optional[str]`) – The config path, a directory where Hydra will search for config files. If config_path is None no directory is added to the search path. Defaults to None.

Return type:

`None`

Training

Training is managed by `train()`, which starts the training process based on the main configuration file (e.g., `conf/config.yaml`).

`autrainer.cli.train(override_kwargs=None, config_name='config', config_path=None)`

Launch a training configuration.

Parameters:

- **override_kwargs** (`Optional[dict]`) – Additional Hydra override arguments to pass to the train script.
- **config_name** (`str`) – The name of the config (usually the file name without the .yaml extension). Defaults to "config".
- **config_path** (`Optional[str]`) – The config path, a directory where Hydra will search for config files. If config_path is None no directory is added to the search path. Defaults to None.

Return type:

`None`

Inference

`inference()` allows for the (sliding window) inference of audio data using a trained model.

Both local paths and Hugging Face Hub links are supported for the model. Hugging Face Hub links are automatically downloaded and cached in the torch cache directory.

The following syntax is supported for Hugging Face Hub links: `hf:repo_id[@revision][:subdir]#local_dir`. This syntax consists of the following components:

- `hf`: The Hugging Face Hub prefix indicating that the model is fetched from the Hugging Face Hub.
- `repo_id`: The repository ID of the model consisting of the user name and the model card name separated by a slash (e.g., `autrainer/example`).
- `revision` (*optional*): The revision as a commit hash, branch name, or tag name (e.g., `main`). If not specified, the latest revision is used.
- `subdir` (*optional*): The subdirectory of the repository containing the model directory (e.g., `AudioModel`). If not specified, the model directory is automatically inferred. If multiple models are present in the `repo_id`, `subdir` must be specified, as the correct model cannot be automatically inferred.

- `local_dir` (*optional*): The local directory to which the model is downloaded to (e.g., `.hf_local`). If not specified, the model is placed in the torch hub cache directory.

For example, to download the model from the repository `autrainer/example` at the revision `main` from the subdirectory `AudioModel` and save it to the local directory `.hf_local`, the following `inference()` CLI wrapper function can be used:

```
import autrainer.cli

autrainer.cli.inference(
    model="hf:autrainer/example@main:AudioModel#.hf_local",
    input="input",
    output="output",
    device="cuda:0",
)
```

> **Tip**
>
> To access private repositories, the environment variable `HF_HOME` should point to the Hugging Face User Access Token.
>
> To use a custom endpoint (e.g., for a self-hosted hub), the environment variable `HF_ENDPOINT` should point to the desired endpoint URL.

To use a local model path, the following `inference()` CLI wrapper function can be used:

```
import autrainer.cli

autrainer.cli.inference(
    model="/path/to/AudioModel",
    input="input",
    output="output",
    device="cuda:0",
)
```

autrainer.cli.inference(*model*, *input*, *output*, *checkpoint='_best'*, *device='cpu'*, *extension='wav'*, *recursive=False*, *embeddings=False*, *update_frequency=1*, *preprocess_cfg='default'*, *window_length=None*, *stride_length=None*, *min_length=None*, *sample_rate=None*, *trust_remote=False*)

Perform inference on a trained model.

If called in a notebook, the function will not raise an error and print the error message instead.

Parameters:

- **model** (`str`) – Local path to model directory or Hugging Face link of the format: *hf:repo_id[@revision][:subdir]#local_dir*. Should contain at least one state subdirectory, the *model.yaml*, *file_handler.yaml*, *target_transform.yaml*, and *inference_transform.yaml* files.
- **input** (`str`) – Path to input directory. Should contain audio files of the specified extension.
- **output** (`str`) – Path to output directory. Output includes a YAML file with predictions and a CSV file with model outputs.
- **checkpoint** (`str`) – Checkpoint to use for evaluation. Defaults to '_best'.
- **device** (`str`) – CUDA-enabled device to use for processing. Defaults to 'cpu'.
- **extension** (`str`) – Type of file to look for in the input directory. Defaults to 'wav'.
- **recursive** (`bool`) – Recursively search for files in the input directory. Defaults to False.
- **embeddings** (`bool`) – Extract embeddings from the model in addition to predictions. For each file, a .pt file with embeddings will be saved. Defaults to False.
- **update_frequency** (`int`) – Frequency of progress bar updates. If 0, the progress bar will be disabled. Defaults to

- **update_frequency** (`int`) – Frequency of progress bar updates. If 0, the progress bar will be disabled. Defaults to 1.
- **preprocess_cfg** (`Optional`[`str`]) – Preprocessing configuration to apply to input. Can be a path to a YAML file or the name of the preprocessing configuration in the local or autrainer 'conf/preprocessing' directory. If "default", the default preprocessing configuration used during training will be applied. If None, no preprocessing will be applied. Defaults to "default".
- **window_length** (`Optional`[`float`]) – Window length for sliding window inference in seconds. If None, the entire input will be processed at once. Defaults to None.
- **stride_length** (`Optional`[`float`]) – Stride length for sliding window inference in seconds. If None, the entire input will be processed at once. Defaults to None.
- **min_length** (`Optional`[`float`]) – Minimum length of audio file to process in seconds. Files shorter than the minimum length are padded with zeros. Sample rate has to be specified for padding. If None, no minimum length is enforced. Defaults to None.
- **sample_rate** (`Optional`[`int`]) – Sample rate of audio files in Hz. Has to be specified for sliding window inference. Defaults to None.
- **trust_remote** (`bool`) – Whether to trust and execute custom Python code from Hugging Face models. Warning: This can be dangerous as it may run arbitrary code on your machine! Defaults to False.

Raises:

CommandLineError – If the model, input, or preprocessing configuration does not exist, or if the device is invalid.

Return type:

`None`

Postprocessing

Postprocessing allows for the summarization, visualization, and aggregation of the training results using `postprocess()`. Several cleanup utilities are provided by `rm_failed()` and `rm_states()`. Manual grouping of the training results can be done using `group()`.

autrainer.cli.postprocess(*results_dir*, *experiment_id*, *max_runs=None*, *aggregate=None*)

Postprocess grid search results.

If called in a notebook, the function will not raise an error and print the error message instead.

Parameters:

- **results_dir** (`str`) – Path to grid search results directory.
- **experiment_id** (`str`) – ID of experiment to postprocess.
- **max_runs** (`Optional`[`int`]) – Maximum number of best runs to plot. Defaults to None.
- **aggregate** (`Optional`[`List`[`List`[`str`]]]) – Configurations to aggregate. One or more of: `VALID_AGGREGATIONS`. Defaults to None.

Raises:

CommandLineError – If the results directory or experiment ID dont exist.

Return type:

`None`

autrainer.cli.rm_failed(*results_dir*, *experiment_id*, *force=False*)

Delete failed runs from an experiment.

If called in a notebook, the function will not raise an error and print the error message instead.

Parameters:

- **results_dir** (`str`) – Path to grid search results directory.
- **experiment_id** (`str`) – ID of experiment to postprocess.
- **force** (`bool`) – Force deletion of failed runs without confirmation. Defaults to False.

Raises:

CommandLineError – If the results directory or experiment ID dont exist.

Return type:

`None`

autrainer.cli.rm_states(*results_dir*, *experiment_id*, *keep_best=True*, *keep_runs=None*, *keep_iterations=None*)

Delete states (.pt files) from an experiment.

If called in a notebook, the function will not raise an error and print the error message instead.

Parameters:

- **results_dir** (`str`) – Path to grid search results directory.
- **experiment_id** (`str`) – ID of experiment to postprocess.
- **keep_best** (`bool`) – Keep best states. Defaults to True.
- **keep_runs** (`Optional[List[str]]`) – Runs to keep. Defaults to None.
- **keep_iterations** (`Optional[List[int]]`) – Iterations to keep. Defaults to None.

Raises:

CommandLineError – If the results directory or experiment ID dont exist.

Return type:

`None`

autrainer.cli.group(*override_kwargs=None*, *config_name='group'*, *config_path=None*)

Launch a manual grouping of multiple grid search results.

Parameters:

- **override_kwargs** (`Optional[dict]`) – Additional Hydra override arguments to pass to the train script.
- **config_name** (`str`) – The name of the config (usually the file name without the .yaml extension). Defaults to "group".
- **config_path** (`Optional[str]`) – The config path, a directory where Hydra will search for config files. If config_path is None no directory is added to the search path. Defaults to None.

Return type:

`None`

Hydra Configurations

autrainer uses Hydra to configure training experiments. All configurations are stored in the `conf/` directory and are defined as YAML files.

Main Configuration

The main entry point of *autrainer* is the `conf/config.yaml` file by default. This file defines the configuration of the training experiments over which a grid search is performed.

conf/config.yaml

```yaml
defaults:
  - _autrainer_
  - _self_

results_dir: results
experiment_id: default
iterations: 5

hydra:
  sweeper:
    params:
      +seed: 1
      +batch_size: 32
      +learning_rate: 0.001
      dataset: ToyTabular-C
      model: ToyFFNN
      optimizer: Adam
```

The main configuration file defines the following parameters:

- `defaults`: A list of default configurations (see Defaults List, Optional Defaults and Overrides, and autrainer Defaults).
- `results_dir`: The directory where the results of the training experiments are stored.
- `experiment_id`: The ID of the experiment.
- `hydra/sweeper/params`: The parameters of the sweep.
  - `<attr>`: An attribute to sweep over from the Defaults List (with comma-separated values, e.g., a list of models).
  - `+<attr>`: An attribute to sweep over that is not in the Defaults List (with comma-separated values, e.g., a list of batch sizes).
- `hydra/sweeper/filters`: A list of filters to filter out unwanted hyperparameter combinations (see Sweeper Plugins).

Some parameters of the main configuration file are outsourced to the _autrainer_.yaml defaults file in order to simplify the configuration.

For more information on configuring Hydra, see the Hydra documentation.

> **Tip**
>
> To use a different configuration file as the entry point for training experiments, use the *-cn/--config-name* argument for the autrainer train CLI command:
>
> ```
> autrainer train -cn some_other_config.yaml
> ```
>
> Alternatively, use the `config_name` parameter for the `train()` CLI wrapper function (without the file extension):
>
> ```
> autrainer.cli.train(config_name="some_other_config")
> ```
>
> For more information on command line flags, see Hydra's command line flags documentation.

Configuration Directories

Configuration files imported through the Defaults List are stored in the `conf/` directory. The configuration files are organized in subdirectories (e.g., `conf/dataset/`, `conf/model/`, `conf/optimizer/`, etc.). This directory structure tells Hydra where to look for the configuration files. The following configuration subdirectories are available:

- `conf/augmentation/`
- `conf/dataset/`

- `conf/model/`
- `conf/optimizer/`
- `conf/plotting`
- `conf/preprocessing/`
- `conf/scheduler/`

Creating Configurations

autrainer provides a number of default configurations for models, datasets, optimizers, etc. that can be used out of the box without creating custom configurations. To use a default configuration e.g., a *MobileNetV3-Large-T* model, add it to the `conf/config.yaml` file:

conf/config.yaml

```yaml
...
hydra
  sweeper:
    params:
      ...
      model: MobileNetV3-Large-T
```

> **Tip**
>
> To discover configurations that are available by default, use the autrainer list CLI command or the `list()` CLI wrapper function. For example, to discover all available *MobileNet* configurations, use the following command with a glob pattern:
>
> ```
> autrainer list model --pattern="MobileNet*"
> ```
>
> Alternatively, use the `list()` CLI wrapper function with the `pattern` parameter:
>
> ```
> autrainer.cli.list(directory="model", pattern="MobileNet*")
> ```

To create a new configuration, create a new YAML file in the appropriate configuration subdirectory. Every configuration file should have:

- A unique file name.
- A unique `id` attribute that matches the file name.
- A `_target_` attribute that specifies the python import path of the class to be instantiated.
- Optional attributes that are passed to the class constructor as keyword arguments.

For example, to create a new model configuration, create a new YAML file in the `conf/model/` directory:

conf/model/MobileNetV3-Large-T.yaml

```yaml
id: MobileNetV3-Large-T
_target_: autrainer.models.TorchvisionModel
torchvision_name: mobilenet_v3_large
transfer: true
transform:
  type: image
```

For more information on how to create custom models, see Models.

> **Tip**
>
> To modify an existing configuration provided by *autrainer*, create a new YAML file with the name in the subdirectory. The new configuration will override the existing configuration.
>
> To easily override a configuration, save a local copy of it using the autrainer show command:
>
> ```
> autrainer show model "MobileNetV3-Large-T" --save
> ```
>
> Alternatively, use the `show()` CLI wrapper function with the `save` parameter:
>
> ```
> autrainer.cli.show(directory="model", id="MobileNetV3-Large-T", save=True)
> ```

Shorthand Syntax

For configurations that are not primitive types (e.g., numbers or strings) and are not included in the Defaults List, shorthand syntax is used to reduce the number of configuration files required. Instead of a configuration file, shorthand syntax configurations can be either a string or a dictionary.

- If the configurations is a string, it is interpreted as the python import path of the class to be instantiated.
- If the configuration is a dictionary, the key is interpreted as the python import path of the class to be instantiated, and its values are passed as keyword arguments to the class constructor.

For example, in a dataset configuration, the `tracking_metric` and the list of `metrics` are specified with shorthand syntax:

conf/dataset/ExampleDataset.yaml

```yaml
id: ExampleDataset
_target_: example_dataset.ExampleDataset
...

tracking_metric: autrainer.metrics.Accuracy
metrics:
  - autrainer.metrics.Accuracy
  - autrainer.metrics.UAR
  - autrainer.metrics.F1
  - custom_metric.CustomMetric:
      param1: 1
      param2: 2
```

Shorthand syntax is used to specify the pipeline of Augmentations, the Loggers, the Metrics for the dataset, and the model as well as Transforms.

Interpolation Syntax

autrainer supports OmegaConf variable interpolation to reference attributes from anywhere in the configuration.

For example, the custom loggers tutorial uses the OmegaConf interpolation syntax to reference the `results_dir` from the main configuration file to set the output directory of the `WandBLogger`:

conf/config.yaml

```yaml
...
loggers:
  - wandb_logger.WandBLogger:
      output_dir: ${results_dir}/.wandb
...
```

For more information on variable interpolation, refer to the OmegaConf documentation.

Defaults List

The defaults list instructs Hydra to import configurations from other YAML files to build the final configuration. For more information on the defaults list, see Hydra's defaults list documentation.

For brevity, the defaults list is outsourced to _autrainer_.yaml defaults file with the following imports:

autrainer.yaml

```yaml
defaults:
  - _self_
  - dataset: ??? # placeholder
  - model: ??? # placeholder
  - optimizer: ??? # placeholder
  - scheduler: None # optional default
  - augmentation: None # optional default
  - plotting: Default # optional default
```

Optional Defaults and Overrides

Optional defaults like `scheduler` and `augmentation` are set to `None` by default and not required to be defined in the main configuration. `None` is a special value that tells Hydra to ignore the default and e.g., not use a scheduler or augmentation for training.

Optional defaults which are not overridden in the `hydra/sweeper/params` configuration, can be overridden using the `override` keyword in the defaults list of the main configuration file. This includes the plotting configuration or Hydra plugins like sweepers and launchers.

For more information on overriding defaults, refer to the Hydra overriding documentation.

autrainer Defaults

The `_autrainer_.yaml` file contains further default configurations to simplify the main configuration, which includes:

- The defaults list and optional defaults list.
- Global default parameters for training, such as the evaluation frequency, save frequency, inference batch size, CUDA-enabled device, etc.
- Hydra configurations for always starting a Hydra multirun (grid search) and setting the output directory and experiment name according to the current configuration.

> **Tip**
>
> Any global default parameter can be overridden in the main configuration file by redefining it.

conf/_autrainer_.yaml

```yaml
defaults:
  - _self_
  - dataset: ToyTabular-C
  - model: ToyFFNN
  - optimizer: Adam
  - scheduler: None
  - augmentation: None
```

```yaml
  - plotting: Default
  - /hydra/callbacks:
      - ConfigDirSnapshot
  - override hydra/sweeper: autrainer_filter_sweeper

training_type: epoch
eval_frequency: 1
save_frequency: ${eval_frequency}
inference_batch_size: ${batch_size}
device: cuda:0

progress_bar: true
continue_training: true
remove_continued_runs: true
save_train_outputs: true
save_dev_outputs: true
save_test_outputs: true

hydra:
  output_subdir: null
  mode: MULTIRUN
  sweep:
    dir: ${results_dir}/${experiment_id}/training/
    subdir: "\
      ${dataset.id}_\
      ${model.id}_\
      ${optimizer.id}_\
      ${learning_rate}_\
      ${batch_size}_\
      ${training_type}_\
      ${iterations}_\
      ${scheduler.id}_\
      ${augmentation.id}_\
      ${seed}"
```

Hydra Plugins

Any Hydra sweeper or launcher plugin can be used to customize the hyperparameter search or parallelize the training jobs.

> **Tip**
>
> Sweepers and launchers can be combined to perform more complex hyperparameter optimizations and parallelize training jobs.

Sweeper Plugins

By default, *autrainer* uses the hydra-filter-sweeper plugin to sweep over hyperparameter configurations. This plugin allows to specify a list of `filters` in the configuration file to filter out unwanted hyperparameter combinations.

> **Tip**
>
> To specify custom `filters`, refer to the filtering out configurations quickstart guide and the hydra-filter-sweeper documentation.

> **Note**
>
> If no `filters` are specified, the plugin will not filter out any configurations and resemble the behavior of the default Hydra basic sweeper plugin.

To perform more complex hyperparameter sweeps, different sweeper plugins can be used.

For example, the Hydra Optuna Sweeper plugin can be used to perform hyperparameter optimization using Optuna.

To install the Optuna Sweeper plugin, run the following command:

```
pip install hydra-optuna-sweeper
```

The following configuration uses the Optuna Sweeper plugin to perform 10 trials with different learning rates:

conf/config.yaml

```yaml
defaults:
  - _autrainer_
  - _self_
  - override hydra/sweeper: optuna # override the sweeper to optuna

results_dir: results
experiment_id: default
iterations: 5

hydra:
  sweeper:
    n_trials: 10 # total number of function evaluations
    n_jobs: 10 # number of parallel jobs
    direction: maximize # direction of optimization (depending on the tracking metric)
    params:
      +seed: 1
      +batch_size: 32
      +learning_rate: range(0.001, 0.1, step=0.001) # range of values
      dataset: ToyTabular-C
      model: ToyFFNN
      optimizer: Adam
```

> **Note**
>
> *autrainer* returns the best validation value of the dataset tracking metric as the objective for optimization. The optimization `direction` attribute in the configuration file can be set to *minimize* or *maximize* according to the dataset tracking metric.

Launcher Plugins

By default, *autrainer* uses the Hydra basic launcher to sequentially launch the jobs defined in the configuration file.

To parallelize the training jobs, different launcher plugins can be used.

For example, the Hydra Submitit Launcher plugin can be used to parallelize the training jobs using Submitit.

To install the Submitit Launcher plugin, run the following command:

```
pip install hydra-submitit-launcher
```

The following configuration uses the Submitit Launcher plugin to parallelize the training jobs:

conf/config.yaml

```yaml
defaults:
  - _autrainer_
  - _self_
  - override hydra/launcher: submitit_slurm # override the launcher to submitit_slurm

results_dir: results
experiment_id: default
iterations: 5

hydra:
```

```yaml
sweeper:
  params:
    +seed: 1
    +batch_size: 32
    +learning_rate: 0.001
    dataset: ToyTabular-C
    model: ToyFFNN
    optimizer: Adam
```

Augmentations

Augmentations are optional and by default not used. This is indicated by the absence of the `augmentation` attribute in the sweeper configuration (implicitly set to a `None` configuration file). To use an augmentation, specify it in the configuration file (`conf/config.yaml`) for the sweeper.

> **Tip**
>
> To create custom augmentations, refer to the custom augmentations tutorial.

Augmentations are specified analogously to transforms using shorthand syntax and have an `order` attribute to define the order of the augmentations. The augmentations are combined with the transform pipeline and sorted based on the order of the augmentations as well as the transforms.

In addition to the order of the augmentation, a seeded probability `p` of applying the augmentation can be specified. The optional `generator_seed` attribute is used to seed the random number generator for the augmentation.

Default Configurations

None

This configuration file is used to indicate that no augmentation is used and serves as a no-op placeholder.

conf/augmentation/None.yaml

```yaml
id: None
_target_: autrainer.augmentations.AugmentationPipeline
pipeline: []
```

Augmentation Pipelines

The `AugmentationManager` is responsible for building the augmentation pipeline.

class **autrainer.augmentations.AugmentationManager(*train_augmentation=None, dev_augmentation=None, test_augmentation=None*)**

Manage the creation of the augmentation pipelines for train, dev, and test sets.

Parameters:

- **train_augmentation** (`Union` [`DictConfig` , `Dict` , `None`]) – Train augmentation configuration.
- **dev_augmentation** (`Union` [`DictConfig` , `Dict` , `None`]) – Dev augmentation configuration.
- **test_augmentation** (`Union` [`DictConfig` , `Dict` , `None`]) – Test augmentation configuration.

get_augmentations()

Get augmentation pipelines for train, dev, and test.

Return type:

`Tuple`[`SmartCompose`, `SmartCompose`, `SmartCompose`]

Returns:

Tuple of augmentation pipelines for train, dev, and test.

The `AugmentationPipeline` class is used to define the configuration and instantiate the augmentation pipeline.

class autrainer.augmentations.AugmentationPipeline(*pipeline*, *generator_seed=0*, *increment=True*)

Initialize an augmentation pipeline.

Parameters:

- **pipeline** (`List`[`Union`[`str`, `Dict`[`str`, `Any`]]]) – The list of augmentations to apply.
- **generator_seed** (`int`) – Seed to pass to each augmentation for reproducibility if the augmentation does not have a seed. Defaults to 0.
- **increment** (`bool`) – Whether to increment the generator seed for each augmentation that does not define its own seed. Defaults to True.

create_pipeline()

Create the composed and ordered augmentation pipeline.

Return type:

`SmartCompose`

Returns:

Composed augmentation pipeline.

Abstract Augmentation

class autrainer.augmentations.AbstractAugmentation(*order=0*, *p=1.0*, *generator_seed=None*, ***kwargs*)

Abstract class for an augmentation.

Parameters:

- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.
- **kwargs** – Additional keyword arguments to store in the object.

Raises:

ValueError – If p is not in the range [0, 1].

offset_generator_seed(*offset*)

Offset the generator seed used to draw the probability of applying the augmentation. Useful for ensuring reproducibility and randomness of augmentations when using multiple workers.

Parameters:

offset (`int`) – Offset to add to the generator seed. Usually the worker index.

Return type:

`None`

`None`

`__call__(item)`

Call the augmentation apply method with probability p.

Parameters:
item (`AbstractDataItem`) – The input data item.

Return type:
`AbstractDataItem`

Returns:
The augmented item if the probability is less than p, otherwise the input item.

abstractmethod `apply(item)`

Apply the augmentation to the input tensor.

Apply is called with probability p.

Parameters:
item (`AbstractDataItem`) – The input data item.

Return type:
`AbstractDataItem`

Returns:
The augmented item.

Augmentation Wrappers

For easier access to common augmentation libraries, *autrainer* provides wrappers for torchaudio, torchvision, torch-audiomentations, and albumentations augmentations.

The underlying augmentation is specified with the `name` attribute, representing the class name of the augmentation. Any further attributes are passed as keyword arguments to the augmentation constructor.

> **Note**
>
> For each augmentation, the probability `p` of applying the augmentation is always available, if the underlying augmentation supports it. If not specified, the default value is 1.0, overriding any existing default value of the library.

Both *torch-audiomentations* and *albumentations* augmentations are optional and can be installed using the following commands.

```
pip install autrainer[albumentations]
pip install autrainer[torch-audiomentations]
```

class autrainer.augmentations.TorchaudioAugmentation(*name*, *order=0*, *p=1.0*, *generator_seed=None*, ***kwargs*)

Wrapper around torchaudio.transforms transforms, which are specified by their class name and keyword arguments.

Important: While the probability of applying the augmentation is deterministic if the generator_seed is set, the actual augmentation applied is not deterministic. This is because the internal random number generator of the augmentation is not seeded.

Parameters:

- **name** (`str`) – Name of the torchaudio augmentation. Must be a valid torchaudio.transforms transform class name.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.
- **kwargs** – Keyword arguments passed to the torchaudio augmentation.

Default Configurations

No default configurations are provided for *torchaudio* augmentations. To discover the available *torchaudio* augmentations, refer to the torchaudio documentation.

class autrainer.augmentations.TorchvisionAugmentation(*name*, *order=0*, *p=1.0*, *generator_seed=None*, ***kwargs*)

Wrapper around torchvision.transforms.v2 transforms, which are specified by their class name and keyword arguments.

Functionals are currently not supported.

Important: While the probability of applying the augmentation is deterministic if the generator_seed is set, the actual augmentation applied is not deterministic. This is because the internal random number generator of the augmentation is not seeded.

Parameters:

- **name** (`str`) – Name of the torchvision augmentation. Must be a valid torchvision.transforms.v2 transform class name.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.
- **kwargs** – Keyword arguments passed to the torchvision augmentation.

Default Configurations

AugMix

conf/augmentation/AugMix.yaml

```yaml
id: AugMix
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.TorchvisionAugmentation:
      name: AugMix
```

GaussianBlur

conf/augmentation/GaussianBlur.yaml

```yaml
id: GaussianBlur
```

```yaml
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.TorchvisionAugmentation:
      name: GaussianBlur
      kernel_size: 3
      sigma: [0.1, 2]
```

RandAugment

conf/augmentation/RandAugment-light.yaml

```yaml
id: RandAugment-light
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.TorchvisionAugmentation:
      name: RandAugment
      num_ops: 2
      magnitude: 0
```

conf/augmentation/RandAugment-medium.yaml

```yaml
id: RandAugment-medium
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.TorchvisionAugmentation:
      name: RandAugment
      num_ops: 2
      magnitude: 15
```

conf/augmentation/RandAugment-strong.yaml

```yaml
id: RandAugment-strong
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.TorchvisionAugmentation:
      name: RandAugment
      num_ops: 2
      magnitude: 20
```

RandGrayscale

conf/augmentation/RandGrayscale.yaml

```yaml
id: RandGrayscale
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.TorchvisionAugmentation:
      name: RandomGrayscale
      p: 0.5
```

class autrainer.augmentations.**AudiomentationsAugmentation**(*name*, *sample_rate=None*, *order=0*, *p=1.0*, *generator_seed=None*, ***kwargs*)

Wrapper around audiomentations transforms, which are specified by their class name and keyword arguments.

Audiomentations operates on numpy arrays, so the input tensor is converted to a numpy array before applying the augmentation, and the output numpy array is converted back to a tensor.

Important: While the probability of applying the augmentation is deterministic if the generator_seed is set, the actual augmentation applied is not deterministic. This is because the internal random number generator of the augmentation is not seeded.

Parameters:

- **name** (`str`) – Name of the torchaudio augmentation. Must be a valid audiomentations transform class name.
- **sample_rate** (`Optional`[`int`]) – The sample rate of the audio data. Should be specified for most audio augmentations. If None, the sample rate is not passed to the augmentation. Defaults to None.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.
- **kwargs** – Keyword arguments passed to the audiomentations augmentation.

Default Configurations

No default configurations are provided for *audiomentations* augmentations. To discover the available *audiomentations* augmentations, refer to the audiomentations documentation.

class autrainer.augmentations.**TorchAudiomentationsAugmentation**(*name*, *sample_rate*, *order=0*, *p=1.0*, *generator_seed=None*, ***kwargs*)

Wrapper around torch_audiomentations transforms, which are specified by their class name and keyword arguments.

Important: While the probability of applying the augmentation is deterministic if the generator_seed is set, the actual augmentation applied is not deterministic. This is because the internal random number generator of the augmentation is not seeded.

Parameters:

- **name** (`str`) – Name of the torchaudio augmentation. Must be a valid torch_audiomentations transform class name.
- **sample_rate** (`int`) – The sample rate of the audio data.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.
- **kwargs** – Keyword arguments passed to the torch_audiomentations augmentation.

Default Configurations

No default configurations are provided for *torch-audiomentations* augmentations. To discover the available *torch-audiomentations* augmentations, refer to the torch-audiomentations documentation.

class autrainer.augmentations.**AlbumentationsAugmentation**(*name*, *order=0*, *p=1.0*, *generator_seed=None*, ***kwargs*)

Wrapper around albumentations transforms, which are specified by their class name and keyword arguments.

Albumentations operates on numpy arrays, so the input tensor is converted to a numpy array before applying the augmentation, and the output numpy array is converted back to a tensor.

Important: While the probability of applying the augmentation is deterministic if the generator_seed is set, the actual augmentation applied is not deterministic. This is because the internal random number generator of the augmentation is not seeded.

Parameters:

- **name** (`str`) – Name of the albumentations augmentation. Must be a valid albumentations transform class name.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.
- **kwargs** – Keyword arguments passed to the albumentations augmentation.

> **Default Configurations**
>
> No default configurations are provided for *albumentations* augmentations. To discover the available *albumentations* augmentations, refer to the albumentations documentation.

Augmentation Graphs

To create more complex augmentation pipelines which may resemble a graph structure, `Sequential` and `Choice` can be used.

> **Tip**
>
> To create custom augmentation graphs, refer to the custom augmentation graphs tutorial.

class autrainer.augmentations.**Sequential**(*sequence*, *order=0*, *p=1.0*, *generator_seed=None*)

Create a fixed sequence of augmentations.

The order of the augmentations in the list is not considered and is placed with respect to the order of the sequence augmentation itself. This means that the sequence of augmentations is applied in the order they are defined in the list and not disrupted by any other transform.

Augmentations in the list must not have a collate function.

Parameters:

- **sequence** (`List`[`Dict`]) – A list of (shorthand syntax) dictionaries defining the augmentation sequence.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

offset_generator_seed(*offset*)

Offset the generator seed used to draw the probability of applying the augmentation. Useful for ensuring

reproducibility and randomness of augmentations when using multiple workers.

Parameters:

offset (`int`) – Offset to add to the generator seed. Usually the worker index.

Return type:

`None`

apply(*item*)

Apply all augmentations in sequence to the input tensor.

Parameters:

item (`AbstractDataItem`) – The input data item.

Return type:

`AbstractDataItem`

Returns:

The augmented item.

class autrainer.augmentations.Choice(*choices*, *weights=None*, *order=0*, *p=1.0*, *generator_seed=None*)

Choose one augmentation from a list of augmentations with a given probability.

The order of the augmentations in the list is not considered and is placed with respect to the order of the choice augmentation itself.

Augmentations in the list must not have a collate function.

Parameters:

- **choices** (`List`[`Dict`]) – A list of (shorthand syntax) dictionaries defining the augmentations to choose from.
- **weights** (`Optional`[`List`[`float`]]) – A list of weights for each choice. If None, all augmentations are assigned equal weights. Defaults to None.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Raises:

- **ValueError** – If choices and weights have different lengths.
- **ValueError** – If any augmentation has a collate function.

offset_generator_seed(*offset*)

Offset the generator seed used to draw the probability of applying the augmentation. Useful for ensuring reproducibility and randomness of augmentations when using multiple workers.

Parameters:

offset (`int`) – Offset to add to the generator seed. Usually the worker index.

Return type:

`None`

apply(*item*)

Choose one augmentation from the list of augmentations based on the given weights.

Parameters:

item (`AbstractDataItem`) – The input data item.

Return type:

`AbstractDataItem`

Returns:

The augmented item.

> **Note**
>
> The order of `Sequential` and `Choice` can be defined in the configuration file by the `order` attribute. However, order attributes of the augmentations within the `Sequential` and `Choice` are ignored. As the augmentations are applied in a scoped manner, their order is determined by the order of the augmentations in the configuration file.

Spectrogram Augmentations

class **autrainer.augmentations.GaussianNoise(*mean=0.0, std=1.0, order=0, p=1.0, generator_seed=None*)**

Add Gaussian noise to the input tensor with mean and standard deviation.

Parameters:

- **mean** (`float`) – The mean of the Gaussian noise. Defaults to 0.0.
- **std** (`float`) – The standard deviation of the Gaussian noise. Defaults to 1.0.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Default Configurations

conf/augmentation/GaussianNoise.yaml

```yaml
id: GaussianNoise
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.GaussianNoise:
      mean: 0.0
      std: 0.001
```

class **autrainer.augmentations.TimeMask(*time_mask, axis, replace_with_zero=True, order=0, p=1.0, generator_seed=None*)**

Mask a random number of time steps.

Important: While the probability of applying the augmentation is deterministic if the generator_seed is set, the actual augmentation applied is not deterministic. This is because the internal random number generator of the augmentation is not seeded.

not seeded.

Parameters:

- **time_mask** (`int`) – maximum time steps in a tensor will be masked.
- **axis** (`int`) – Time axis. If the image is torch Tensor, it is expected to have [C, H, W] shape, then H is assumed to be axis 0, and W is axis 1.
- **replace_with_zero** (`bool`) – Fill the mask either with a tensor mean, or 0's. Defaults to True.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Default Configurations

conf/augmentation/TimeMask.yaml

```yaml
id: TimeMask
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.TimeMask:
      time_mask: 80
      axis: 0
```

class **autrainer.augmentations.FrequencyMask**(***freq_mask***, ***axis***, ***replace_with_zero***=*True*, ***order***=*0*, ***p***=*1.0*, ***generator_seed***=*None*)

Mask a random number of frequency steps.

Important: While the probability of applying the augmentation is deterministic if the generator_seed is set, the actual augmentation applied is not deterministic. This is because the internal random number generator of the augmentation is not seeded.

Parameters:

- **freq_mask** (`int`) – maximum frequency steps in a tensor will be masked.
- **axis** (`int`) – Frequency axis. If the image is torch Tensor, it is expected to have [C, H, W] shape, then H is assumed to be axis 0, and W is axis 1.
- **replace_with_zero** (`bool`) – Fill the mask either with a tensor mean, or 0's. Defaults to True.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Default Configurations

conf/augmentation/FrequencyMask.yaml

```yaml
id: FrequencyMask
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0
```

```yaml
generator_seed: 0

pipeline:
  - autrainer.augmentations.FrequencyMask:
      freq_mask: 10
      axis: 1
```

class autrainer.augmentations.TimeShift(*axis*, *time_steps=0*, *order=0*, *p=1.0*, *generator_seed=None*)

Shift the input tensor along the time axis.

Parameters:

- **axis** (`int`) – Time axis. If the image is torch Tensor, it is expected to have [C, H, W] shape, then H is assumed to be axis 0, and W is axis 1.
- **time_steps** (`int`) – maximum time steps a tensor will shifted forward or backward. Defaults to 0.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Default Configurations

conf/augmentation/TimeShift.yaml

```yaml
id: TimeShift
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.TimeShift:
      time_steps: 20
      axis: 0
```

class autrainer.augmentations.TimeWarp(*axis*, *W=10*, *order=0*, *p=1.0*, *generator_seed=None*)

A random point along the time axis passing through the center of the image within the time steps (W, tau - W) is to be warped either to the left or right by a distance w chosen from a uniform distribution from 0 to the time warp parameter W along that line.

Parameters:

- **axis** (`int`) – Time axis. If the image is torch Tensor, it is expected to have [C, H, W] shape, then H is assumed to be axis 0, and W is axis 1.
- **W** (`int`) – Bound for squishing/stretching. Defaults to 10.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Default Configurations

conf/augmentation/TimeWarp.yaml

```yaml
id: TimeWarp
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.TimeWarp:
      W: 30
      axis: 0
```

class **autrainer.augmentations.SpecAugment(*time_mask=10, freq_mask=10, W=50, order=0, p=1.0, generator_seed=None*)**

SpecAugment augmentation. A combination of time warp, frequency masking, and time masking.

Important: While the probability of applying the augmentation is deterministic if the generator_seed is set, the actual augmentation applied is not deterministic. This is because the internal random number generator of the augmentation is not seeded.

For more information, see: https://arxiv.org/abs/1904.08779

This implementation differs from PyTorch, as they apply TimeStrech instead of TimeWarp. For more information, see: https://pytorch.org/audio/master/tutorials/audio_feature_augmentation_tutorial.html#specaugment

Parameters:

- **time_mask** (`int`) – maximum time steps in a tensor will be masked. Defaults to 10.
- **freq_mask** (`int`) – maximum frequency steps in a tensor will be masked. Defaults to 10.
- **W** (`int`) – Bound for squishing/stretching the time axis. Defaults to 50.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Default Configurations

conf/augmentation/SpecAugment.yaml

```yaml
id: SpecAugment
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.SpecAugment:
      freq_mask: 10
      time_mask: 80
      W: 30
      p: 0.5
```

Augmentations with Collate Functions

For augmentations that require a collate function, an optional `get_collate_fn` method can be implemented. This method is

For augmentations that require a collate function, an optional `get_collate_fn` method can be implemented. This method is used to retrieve the collate function from the augmentation if it is present.

> **Tip**
>
> To create custom augmentations with collate functions, refer to the custom augmentations tutorial.

The signature of the `get_collate_fn` method should be as follows and return a collate function taking in a list of `AbstractDataItem` objects and returning a single `AbstractDataBatch` object.

example_augmentation.ExampleCollateAugmentation

```python
class ExampleCollateAugmentation(AbstractAugmentation):
    def get_collate_fn(
        self,
        data: "AbstractDataset",
        default: Callable,
    ) -> Callable:
        return DataBatch.collate
```

> **Note**
>
> Only one collate function can be used in each transform pipeline. If multiple collate functions are defined, the last one in the pipeline (defined by the order of the transforms) is used.

Both `CutMix` and `MixUp` augmentations require a collate function and operate on the batch level. This means, that the collate function is applied to the batch of samples, rather than individual samples, and the probability of applying the augmentation acts on the batch level as well.

class autrainer.augmentations.CutMix(*alpha=1.0, order=0, p=1.0, generator_seed=None*)

CutMix augmentation. As CutMix utilizes a collate function, the probability of applying the augmentation is drawn for each batch.

Parameters:

- **alpha** (`float`) – Hyperparameter of the Beta distribution. Defaults to 1.0.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Default Configurations

conf/augmentation/CutMix.yaml

```yaml
id: CutMix
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.CutMix:
      alpha: 0.8
```

class autrainer.augmentations.**MixUp**(*alpha=1.0*, *order=0*, *p=1.0*, *generator_seed=None*)

MixUp augmentation. As MixUp utilizes a collate function, the probability of applying the augmentation is drawn for each batch.

Parameters:

- **alpha** (`float`) – Hyperparameter of the Beta distribution. Defaults to 1.0.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 0.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Default Configurations

conf/augmentation/MixUp.yaml

```yaml
id: MixUp
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.MixUp:
      alpha: 0.8
```

Miscellaneous Augmentations

class autrainer.augmentations.**SampleGaussianWhiteNoise**(*snr_df*, *snr_col*, *sample_seed=None*, *order=101*, *p=1.0*, *generator_seed=None*)

Sample-level gaussian white noise augmentation based on SNR values.

Parameters:

- **snr_df** (`str`) – Path to a CSV file containing SNR values for each sample. Index of the CSV file must match the index of the dataset.
- **snr_col** (`str`) – Name of the column containing the SNR values.
- **sample_seed** (`int`) – Seed for the random number generator used for sampling the noise. If a seed is provided, a consistent augmentation is applied to the same sample. Defaults to None.
- **order** (`int`) – The order of the augmentation in the transformation pipeline. Defaults to 101.
- **p** (`float`) – The probability of applying the augmentation. Defaults to 1.0.
- **generator_seed** (`Optional`[`int`]) – The initial seed for the internal random number generator drawing the probability. If None, the generator is not seeded. Defaults to None.

Default Configurations

conf/augmentation/SampleGaussianWhiteNoise.yaml

```yaml
id: SampleGaussianWhiteNoise
_target_: autrainer.augmentations.AugmentationPipeline

generator_seed: 0

pipeline:
  - autrainer.augmentations.SampleGaussianWhiteNoise:
      snr_df: ???
```

```
9       snr_col: ???
10      sample_seed: ???
```

Core

Core provides various utilities and entry points for the *autrainer* framework.

Entry Point

The main training entry point for *autrainer*.

`autrainer.main(config_name, config_path=None, version_base=None)`

Hydra main decorator with additional *autrainer* configs.

The *conf* directory in the current working directory is always added to the search path if it exists. The current working directory is also added to the Python path.

Parameters:

- **config_name** (`str`) – The name of the config (usually the file name without the .yaml extension).
- **config_path** (`Optional`[`str`]) – The config path, a directory where Hydra will search for config files. If config_path is None no directory is added to the search path. Defaults to None.
- **version_base** (`Optional`[`str`]) – Hydra version base. Defaults to None.

Instantiation

Instantiation functions provide wrappers around Hydra object instantiation, providing additional type safety and Shorthand Syntax support.

`autrainer.instantiate(config, instance_of=None, convert=None, recursive=False, **kwargs)`

Instantiate an object from a configuration Dict or DictConfig.

The config must contain a *_target_* field that specifies a relative import path to the object to instantiate. If *_target_* is None, returns None.

Parameters:

- **config** (`Union`[`DictConfig`, `Dict`]) – The configuration to instantiate.
- **instance_of** (`Optional`[`Type`[`TypeVar`(`T`)]]) – The expected type of the instantiated object. Defaults to None.
- **convert** (`Optional`[`HydraConvertEnum`]) – The conversion strategy to use, one of HydraConvertEnum. Convert is only used if the config does not have a *_convert_* attribute. If None, uses HydraConvertEnum.ALL. Defaults to None.
- **recursive** (`bool`) – Whether to recursively instantiate objects. Recursive is only used if the config does not have a *_recursive_* field. Defaults to False.
- ****kwargs** – Additional keyword arguments to pass to the object.

Raises:

- **ValueError** – If the config does not have a *_target_* field.
- **ValueError** – If the instantiated object is not an instance of *instance_of* and *instance_of* is provided.

Return type:

`TypeVar`(`T`)

Returns:

The instantiated object

The instantiated object.

autrainer.instantiate_shorthand(*config*, *instance_of=None*, *convert=None*, *recursive=False*, ***kwargs*)

Instantiate an object from a shorthand configuration.

A shorthand config is either a string or a dictionary with a single key. If config is a string, it should be a python import path. If config is a dictionary, the key should be a python import path and the value should be a dictionary of keyword arguments.

Parameters:

- **config** (`Union`[`str`, `DictConfig`, `Dict`]) – The config to instantiate.
- **instance_of** (`Optional`[`Type`[`TypeVar`(`T`)]]) – The expected type of the instantiated object. Defaults to None.
- **convert** (`Optional`[`HydraConvertEnum`]) – The conversion strategy to use, one of HydraConvertEnum. Convert is only used if the config does not have a *_convert_* attribute. If None, uses HydraConvertEnum.ALL. Defaults to None.
- **recursive** (`bool`) – Whether to recursively instantiate objects. Recursive is only used if the config does not have a *_recursive_* field. Defaults to False.
- ****kwargs** – Additional keyword arguments to pass to the object.

Raises:

ValueError – If the config is empty (None or an empty string/dictionary).

Return type:

`TypeVar`(`T`)

Returns:

The instantiated object.

Data Items and Batches

autrainer provides a set of classes to represent data items and batches of data items. The `DataItem` class represents individual data *instances*. These classes (or *structs*) are meant to hold all important attributes needed by `AbstractModel` objects that are compatible with a particular dataset. In addition they hold attributes needed by *autrainer*'s utilities, such as the *index* corresponding to each *instance* (where we assume that each `AbstractDataset` is an ordered set of instances).

class autrainer.core.structs.AbstractDataItem(*features*, *target*, *index*)

Abstract data item class for a single sample.

Parameters:

- **features** (`Tensor`) – Tensor of input features.
- **target** (`Union`[`int`, `float`, `List`[`int`], `List`[`float`], `ndarray`]) – Target value for the input features.
- **index** (`int`) – Index of the data sample.

class autrainer.core.structs.DataItem(*features*, *target*, *index*)

Data item for a single sample.

Parameters:

- **features** (`Tensor`) – Tensor of input features.
- **target** (`Union`[`int`, `float`, `List`[`int`], `List`[`float`], `ndarray`]) – Target value for the input features.
- **index** (`int`) – Index of the data sample.

Additionally, *autrainer* provides a set of classes that hold *batches* of individual data instances. They provide an implementation of the `collate_fn()` which is used to collate the individual data items into a batch of data items and must be passed to the `torch.utils.data.DataLoader`.

class autrainer.core.structs.AbstractDataBatch(*features*, *target*, *index*)

Abstract data batch class for a batch of samples.

Parameters:

- **features** (`Tensor`) – Tensor of input features.
- **target** (`Tensor`) – Tensor of target values for the input features.
- **index** (`Tensor`) – Tensor of indices for the data samples.

abstractmethod to(*device*, ***kwargs*)

Move the features, target, and additional data to a device.

Parameters:

- **device** (`device`) – Device to move the data to.
- **kwargs** (`dict`) – Additional keyword arguments passed to the *to* method.

Return type:

`None`

classmethod collate(*items*)

Collate a list of data items into a data batch.

Parameters:

items (`List`[`TypeVar`(`ItemType`, bound= AbstractDataItem)]) – List of data items to collate.

Return type:

`AbstractDataBatch`[`TypeVar`(`ItemType`, bound= AbstractDataItem)]

Returns:

Collated data batch.

class autrainer.core.structs.DataBatch(*features*, *target*, *index*)

Data batch for a batch of samples.

Parameters:

- **features** (`Tensor`) – Tensor of input features.
- **target** (`Tensor`) – Tensor of target values for the input features.
- **index** (`Tensor`) – Tensor of indices for the data samples.

to(*device*, ***kwargs*)

Move the features, target, and additional data to a device.

Parameters:

- **device** (`device`) – Device to move the data to.
- **kwargs** (`dict`) – Additional keyword arguments passed to the *to* method.

Return type:

`None`

> **Warning**
>
> `features`, `target`, and `index` are the three **reserved** attributes that every object derived from `AbstractDataItem` **must** include. Moreover, every object derived from `AbstractModel` **must** include `features` as the first argument (after *self*) of its `forward()` method.

Utils

Utils provide various helpers for I/O, logging, timing, and hardware information.

class autrainer.core.utils.Bookkeeping(*output_directory*, *file_handler_path=None*)

Bookkeeping to handle general disk operations and interactions.

Parameters:

- **output_directory** (`str`) – Output directory to save files to.
- **file_handler_path** (`Optional`[`str`]) – Path to save the log file to. Defaults to None.

log(*message*, *level=20*)

Log a message.

Parameters:

- **message** (`str`) – Message to log.
- **level** (`int`) – Logging level. Defaults to logging.INFO.

Return type:

`None`

log_to_file(*message*, *level=20*)

Log a message to the file handler.

Parameters:

- **message** (`str`) – Message to log.
- **level** (`int`) – Logging level. Defaults to logging.INFO.

Return type:

`None`

create_folder(*folder_name*, *path=''*)

Create a new folder in the output directory.

Parameters:

- **folder_name** (`str`) – Name of the folder to create.
- **path** (`str`) – Subdirectory to create the folder in. Defaults to "".

Return type:

`None`

save_model_summary(*model*, *shape*, *device*, *filename*)

Save a model summary to a file.

Parameters:

- **model** (`Module`) – Model to summarize.
- **shape** (`Tuple`[`int`, `...`]) – Shape of the input to the model.
- **filename** (`str`) – Name of the file to save the summary to.

Return type:

`None`

save_state(*obj*, *filename*, *path=''*)

Save the state of an object.

Parameters:

- **obj** (`Union`[`Module`, `Optimizer`, `LRScheduler`, `None`]) – Object to save the state of. If None, do nothing.
- **filename** (`str`) – Name of the file to save the state to.
- **path** (`str`) – Subdirectory to save the state to. Defaults to “”.

Raises:

TypeError – If the object type is not supported.

Return type:

`None`

load_state(*obj*, *filename*, *path=''*)

Load the state of an object.

Parameters:

- **obj** (`Union`[`Module`, `Optimizer`, `LRScheduler`]) – Object to load the state into.
- **filename** (`str`) – Name of the file to load the state from.
- **path** (`str`) – Subdirectory to load the state from. Defaults to “”.

Raises:

- **TypeError** – If the object type is not supported.
- **FileNotFoundError** – If the file is not found.

Return type:

`None`

save_audobject(*obj*, *filename*, *path=''*)

Save an audobject.Object to disk.

Parameters:

- **obj** (`Object`) – Object to save.
- **filename** (`str`) – Name of the file to save the object to.
- **path** (`str`) – Subdirectory to save the object to. Defaults to “”.

Raises:

TypeError – If the object type is not supported.

Return type:

`None`

save_results_dict(*results_dict*, *filename*, *path=''*)

Save a results dictionary to disk.

Parameters:

- **results_dict** (`Dict[str, float]`) – Dictionary of metric names and values to save.
- **filename** (`str`) – Name of the file to save the results to.
- **path** (`str`) – Subdirectory to save the results to. Defaults to "".

Return type:

`None`

save_results_df(*results_df*, *filename*, *path=''*)

Save a results DataFrame to disk.

Parameters:

- **results_df** (`DataFrame`) – DataFrame to save.
- **filename** (`str`) – Name of the file to save the results to.
- **path** (`str`) – Subdirectory to save the results to. Defaults to "".

Return type:

`None`

save_results_np(*results_np*, *filename*, *path=''*)

Save a results numpy array to disk.

Parameters:

- **results_np** (`ndarray`) – Numpy array to save.
- **filename** (`str`) – Name of the file to save the results to.
- **path** (`str`) – Subdirectory to save the results to. Defaults to "".

Return type:

`None`

save_best_results(*metrics*, *filename*, *metric_fns*, *tracking_metric_fn*, *path=''*)

Save the best results to disk.

Parameters:

- **metrics** (`DataFrame`) – DataFrame of metrics to save.
- **filename** (`str`) – Name of the file to save the best results to.
- **metric_fns** (`List[AbstractMetric]`) – List of metric functions to get the best results from.
- **tracking_metric_fn** (`AbstractMetric`) – Tracking metric function to get the best iteration from.
- **path** (`str`) – Subdirectory to save the best results to. Defaults to "".

Return type:

`None`

class autrainer.core.utils.Timer(*output_directory*, *timer_type*)

Timer to measure time of different parts of the training process.

Parameters:

- **output_directory** (`str`) – Directory to save the timer.yaml file to.
- **timer_type** (`str`) – Name of the timer.

start()

Start the timer.

Return type:
`None`

stop()

Stop the timer.

Raises:
ValueError – If the timer was not started.

Return type:
`None`

get_time_log()

Get the time log.

Return type:
`list`

Returns:
List of times.

get_mean_seconds()

Get the mean time in seconds.

Return type:
`float`

Returns:
Mean time in seconds.

get_total_seconds()

Get the total time in seconds.

Return type:
`float`

Returns:
Total time in seconds.

classmethod pretty_time(*seconds*)

Convert seconds to a pretty string.

Parameters:
seconds (`float`) – Time in seconds.

Return type:
`str`

Returns:

Time in a pretty string format.

save(*path=''*)

Save and append the timer to timer.yaml.

Parameters:

path (`str`) – Subdirectory to save the timer.yaml file to relative to the output directory. Defaults to "".

Return type:

`dict`

Returns:

Dictionary with mean and total time in seconds and pretty format.

autrainer.core.utils.get_hardware_info(*device*)

Get hardware information of the current system.

Parameters:

device (`device`) – Device to get the hardware information from.

Return type:

`dict`

Returns:

Dictionary containing system and GPU information.

autrainer.core.utils.save_hardware_info(*output_directory*, *device*)

Save hardware information to a hardware.yaml file.

Parameters:

- **output_directory** (`str`) – Directory to save the hardware information to.
- **device** (`device`) – Device to get the hardware information from.

Return type:

`None`

autrainer.core.utils.set_seed(*seed*)

Set a global seed for reproducibility for random, numpy, and torch.

If CUDA is available, set the seed for CUDA and cuDNN as well.

Parameters:

seed (`int`) – Seed to set.

Return type:

`None`

autrainer.core.utils.silence()

Context manager to suppress stdout and stderr.

Yields:

None.

Return type:

`Generator`[`None`, `None`, `None`]

Plotting

Plotting provides a simple interface to plot metrics of a single run during Training as well as multiple runs during Postprocessing.

> **Tip**
>
> To create custom plotting configurations, refer to the custom plotting configurations tutorial.

By default, training plots are saved as *png* files for each metric. This can optionally be extended to any format supported by Matplotlib and additionally pickled for further processing.

> **Note**
>
> Plots are fully customizable by providing Matplotlib rcParams in a custom plotting configuration.

class **autrainer.core.plotting.PlotBase(*output_directory*, *training_type*, *figsize*, *latex*, *filetypes*, *pickle*, *context*, *palette*, *replace_none*, *add_titles*, *add_xlabels*, *add_ylabels*, *rcParams*)**

Base class for plotting.

Parameters:

- **output_directory** (`str`) – Output directory to save plots to.
- **training_type** (`str`) – Type of training in [“Epoch”, “Step”].
- **figsize** (`tuple`) – Figure size in inches.
- **latex** (`bool`) – Whether to use LaTeX in plots. Requires the *latex* package. To install all necessary dependencies, run: *pip install autrainer[latex]*.
- **filetypes** (`list`) – Filetypes to save plots as.
- **pickle** (`bool`) – Whether to save additional pickle files of the plots.
- **context** (`str`) – Context for seaborn plots.
- **palette** (`str`) – Color palette for seaborn plots.
- **replace_none** (`bool`) – Whether to replace “None” in labels with “~”.
- **add_titles** (`bool`) – Whether to add titles to plots.
- **add_xlabels** (`bool`) – Whether to add x-labels to plots.
- **add_ylabels** (`bool`) – Whether to add y-labels to plots.
- **rcParams** (`dict`) – Additional Matplotlib rcParams to set.

save_plot(*fig*, *name*, *path*='', *close*=*True*, *tight_layout*=*True*)

Save a plot to the output directory.

Parameters:

- **fig** (`Figure`) – Matplotlib figure to save.
- **name** (`str`) – Name of the plot.
- **path** (`str`) – Path to save the plot to relative to the output directory.
- **close** (`bool`) – Whether to close the figure after saving.
- **tight_layout** (`bool`) – Whether to apply tight layout to the plot.

Return type:

`None`

***class* autrainer.core.plotting.PlotMetrics(*output_directory*, *training_type*, *figsize*, *latex*, *filetypes*, *pickle*, *context*, *palette*, *replace_none*, *add_titles*, *add_xlabels*, *add_ylabels*, *rcParams*, *metric_fns*)**

Plot the metrics of one or multiple runs.

Parameters:

- **output_directory** (`str`) – Output directory to save plots to.
- **training_type** (`str`) – Type of training in ["Epoch", "Step"].
- **figsize** (`tuple`) – Figure size in inches.
- **latex** (`bool`) – Whether to use LaTeX in plots. Requires the *latex* package. To install all necessary dependencies, run: *pip install autrainer[latex]*.
- **filetypes** (`list`) – Filetypes to save plots as.
- **pickle** (`bool`) – Whether to save additional pickle files of the plots.
- **context** (`str`) – Context for seaborn plots.
- **palette** (`str`) – Color palette for seaborn plots.
- **replace_none** (`bool`) – Whether to replace "None" in labels with "~".
- **add_titles** (`bool`) – Whether to add titles to plots.
- **add_xlabels** (`bool`) – Whether to add x-labels to plots.
- **add_ylabels** (`bool`) – Whether to add y-labels to plots.
- **rcParams** (`dict`) – Additional Matplotlib rcParams to set.
- **metric_fns** (`List`[`AbstractMetric`]) – List of metrics to use for plotting.

Default Configurations

Default

conf/plotting/Default.yaml

```yaml
figsize: [10, 5]
latex: false
filetypes: [png]
pickle: false
context: notebook
palette: colorblind
replace_none: false
add_titles: true
add_xlabels: true
add_ylabels: true

rcParams:
  legend.fontsize: 9
```

plot_run(*metrics*, *std_scale=0.1*)

Plot the metrics of a single run.

Parameters:

- **metrics** (`DataFrame`) – DataFrame containing the metrics.
- **std_scale** (`float`) – Scale factor for the standard deviation. Defaults to 0.1.

Return type:

`None`

plot_metric(*metrics*, *metric*, *metrics_std=None*, *std_scale=0.1*, *max_runs=None*)

Plot a single metric of multiple runs.

Parameters:

- **metrics** (`DataFrame`) – DataFrame containing the metrics.
- **metric** (`str`) – Metric to plot.
- **metrics_std** (`Optional`[`DataFrame`]) – DataFrame containing the standard deviations. Defaults to None.
- **std_scale** (`float`) – Scale factor for the standard deviation. Defaults to 0.1.
- **max_runs** (`Optional`[`int`]) – Maximum number of best runs to plot. If None, all runs are plotted. Defaults to None.

Return type:

`None`

plot_aggregated_bars(*metrics_df*, *metric*, *subplots_by=0*, *group_by=1*, *split_subgroups=True*)

Plot aggregated bar plots for a metric.

Generate a bar plots from the metrics_df, which are divided by the “subplots_by” column, further grouped according to the “group_by” column. If “split_subgroups” is set to true, each group is further split into subgroups based on what comes after a potential “-” in the “group_by” entry. Finally the “metric” entries are averaged to create the bars and the standard deviation is shown as error bars.

Parameters:

- **metrics_df** (`DataFrame`) – DataFrame containing the metrics.
- **metric** (`str`) – Metric to plot.
- **subplots_by** (`int`) – Column to group the subplots by.
- **group_by** (`int`) – Column to group the data by.
- **split_subgroups** (`bool`) – Whether to split subgroups.

Constants

autrainer provides a set of constants singletons to control naming, training, and exporting configurations at runtime.

class autrainer.core.constants.AbstractConstants

Abstract constants singleton class for managing the configurations of *autrainer*.

class autrainer.core.constants.NamingConstants

Singleton for managing the naming configurations of *autrainer*.

property NAMING_CONVENTION*: List[str]*

Get the naming convention of runs. Defaults to `["dataset", "model", "optimizer", "learning_rate", "batch_size", "training_type", "iterations", "scheduler", "augmentation", "seed"]`.

Returns:

Naming convention of runs.

property **INVALID_AGGREGATIONS**: *List[str]*

Get the invalid aggregations for postprocessing. Defaults to `["training_type"]`.

Returns:

Invalid aggregations for postprocessing.

property **VALID_AGGREGATIONS**: *List[str]*

Get the valid aggregations for postprocessing. Defaults to `["dataset", "model", "optimizer", "learning_rate", "batch_size", "iterations", "scheduler", "augmentation", "seed"]` (the naming convention without the invalid aggregations).

Returns:

Valid aggregations for postprocessing.

property **CONFIG_DIRS**: *List[str]*

Get the configuration directories for Hydra configurations. Defaults to `["augmentation", "dataset", "model", "optimizer", "plotting", "preprocessing", "scheduler"]`.

Returns:

Configuration directories for Hydra configurations.

class **autrainer.core.constants.TrainingConstants**

Singleton for managing the training configurations of *autrainer*.

property **TASKS**: *List[str]*

Get the supported training tasks. Defaults to `["classification", "ml-classification", "regression", "mt-regression"]`.

Returns:

Supported training tasks.

class **autrainer.core.constants.ExportConstants**

Singleton for managing the export and logging configurations of *autrainer*.

property **LOGGING_DEPTH**: *int*

Get the depth of logging for configuration parameters. Defaults to `2`.

Returns:

Depth of logging for configuration parameters.

property **IGNORE_PARAMS**: *List[str]*

Get the ignored configuration parameters for logging. Defaults to `["results_dir", "experiment_id", "model.dataset", "training_type", "save_frequency", "dataset.metrics", "plotting", "model.transform", "dataset.transform", "augmentation.steps", "loggers", "progress_bar", "continue_training", "remove_continued_runs", "save_train_outputs", "save_dev_outputs", "save_test_outputs"]`.

Returns:

Ignored configuration parameters for logging.

property **ARTIFACTS**: *List[str | Dict[str, str]]*

Get the artifacts to log for runs. Defaults to `["model_summary.txt", "metrics.csv", {"config.yaml": ".hydra"}]`.

Returns:

Artifacts to log for runs.

Criterions

Criterions are specified in the Dataset configuration with Shorthand Syntax and are used to calculate the loss of the model.

> **Tip**
>
> To create custom criterions, refer to the custom criterions tutorial.

> **Note**
>
> The `reduction` attribute of each criterion is automatically set to `"none"` during training, validation, and testing. This allows the per-example loss to be reported directly, without the need for re-calculating the loss for logging purposes.
>
> This is handled automatically during the instantiation of the criterion.

Criterion Wrappers

As the DataLoader may automatically cast to the wrong type, some loss functions need to be wrapped to cast the model outputs and targets to the correct types. For more information see this discussion.

autrainer provides the following wrappers:

class autrainer.criterions.**CrossEntropyLoss**(**args*, ***kwargs*)

forward(*x*, *y*)

Wrapper for *torch.nn.CrossEntropyLoss.forward*.

Converts the targets to *long* if it is a 1D tensor.

Parameters:

- **x** (`Tensor`) – Batched model outputs.
- **y** (`Tensor`) – Targets.

Return type:

`Tensor`

Returns:

Loss.

class autrainer.criterions.**BCEWithLogitsLoss**(**args*, ***kwargs*)

forward(*x*, *y*)

Wrapper for *torch.nn.BCEWithLogitsLoss.forward*.

Parameters:

- **x** (`Tensor`) – Batched model outputs.
- **y** (`Tensor`) – Targets.

Return type:

`Tensor`

Returns:

Loss.

class **autrainer.criterions.MSELoss(**args*, ***kwargs*)**

forward(*x*, *y*)

Wrapper for *torch.nn.MSELoss.forward*.

Squeezes the model outputs along the last dimension to match the shape of the targets. Converts the targets to *float*.

Parameters:

- **x** (`Tensor`) – Batched model outputs.
- **y** (`Tensor`) – Targets.

Return type:

`Tensor`

Returns:

Loss.

Balanced and Weighted Criterions

For imbalanced datasets, it is often beneficial to use a balanced or weighted loss function. Balanced loss functions automatically adjust the loss for each class or target based on their frequency using a setup function. Weighted loss functions allow for the manual specification of weights for each class or target.

class **autrainer.criterions.BalancedCrossEntropyLoss(**args*, ***kwargs*)**

setup(*data*)

Calculate balanced weights for the dataset based on the target frequency in the training set.

Parameters:

data (AbstractDataset) – Instance of the dataset.

Return type:

`None`

class **autrainer.criterions.WeightedCrossEntropyLoss(*class_weights*, ***kwargs*)**

Wrapper for *torch.nn.CrossEntropyLoss* with manual class weights.

The class weights are automatically normalized to sum up to the number of classes.

Parameters:

- **class_weights** (`Dict`[`str`, `float`]) – Dictionary with class weights corresponding to the target labels and their respective weights.
- ****kwargs** – Additional keyword arguments passed to *torch.nn.CrossEntropyLoss*.

setup(*data*)

Calculate the class weights based on the provided dictionary.

Parameters:

data (AbstractDataset) – Instance of the dataset.

Return type:

`None`

class autrainer.criterions.BalancedBCEWithLogitsLoss(*weight=None*, *reduction='mean'*)

Balanced version of *torch.nn.BCEWithLogitsLoss*.

pos_weight is not supported, as the weights are calculated based on the target frequency in the training set.

Parameters:

- **weight** (`Optional`[`Tensor`]) – A manual rescaling weight given to the positive class. Defaults to None.
- **reduction** (`str`) – Specifies the reduction to apply to the output. Defaults to 'mean'.

setup(*data*)

Calculate balanced weights for the dataset based on the target frequency in the training set.

Parameters:

data (AbstractDataset) – Instance of the dataset.

Return type:

`None`

forward(*x*, *y*)

Wrapper for *torch.nn.BCEWithLogitsLoss.forward* with balanced weights.

Parameters:

- **x** (`Tensor`) – Batched model outputs.
- **y** (`Tensor`) – Targets.

Return type:

`Tensor`

Returns:

Loss.

class autrainer.criterions.WeightedBCEWithLogitsLoss(*class_weights*, ***kwargs*)

Wrapper for *torch.nn.BCEWithLogitsLoss* with manual class weights.

The class weights are automatically normalized to sum up to the number of classes.

Parameters:

- **class_weights** (`Dict`[`str`, `float`]) – Dictionary with class weights corresponding to the target labels and their respective weights.
- ****kwargs** – Additional keyword arguments passed to *torch.nn.BCEWithLogitsLoss*.

setup(*data*)

Calculate the class weights based on the provided dictionary.

Parameters:

data (AbstractDataset) – Instance of the dataset.

data (AbstractDataset) – Instance of the dataset.

Return type:

`None`

class autrainer.criterions.WeightedMSELoss(*target_weights*, ***kwargs*)

Wrapper for *torch.nn.MSELoss* with manual target weights intended for multi-target regression tasks.

The target weights are automatically normalized to sum up to the number of targets.

Parameters:

- **target_weights** (`Dict`[`str`, `float`]) – Dictionary with target weights corresponding to the target labels and their respective weights.
- ****kwargs** – Additional keyword arguments passed to *torch.nn.MSELoss*.

setup(*data*)

Calculate the target weights based on the provided dictionary.

Parameters:

data (AbstractDataset) – Instance of the dataset.

Return type:

`None`

forward(*x*, *y*)

Wrapper for *torch.nn.MSELoss.forward* with manual target weights.

Parameters:

- **x** (`Tensor`) – Batched model outputs.
- **y** (`Tensor`) – Targets.

Return type:

`Tensor`

Returns:

Loss.

Torch Criterions

Other torch criterions, such as `torch.nn.HuberLoss` or `torch.nn.SmoothL1Loss`, can be specified using Shorthand Syntax in the dataset configuration, analogous to the criterion wrappers.

> **Note**
>
> It may be necessary to wrap criterions similar to `autrainer.criterions.CrossEntropyLoss` or `autrainer.criterions.MSELoss` to cast the model outputs and targets to the correct types.

Criterion Setup

Criterions can optionally provide a `setup()` method which is called after the criterion is initialized and takes the dataset instance as an argument. This can be used to set up additional parameters, such as class weights for imbalanced datasets.

```
example_loss.ExampleLoss
```

```python
class ExampleLoss(torch.nn.modules.loss._Loss):
    def setup(self, data: "AbstractDataset") -> None: ...
```

Datasets

autrainer provides a number of different audio-specific datasets, base datasets for different tasks, and toy datasets for testing purposes. To ensure consistency across different data formats and manage multiple data types, all datasets should follow a standardized structure.

> **Tip**
>
> To create custom datasets, refer to the custom datasets tutorial.

In addition to the common attributes like `id`, `_target_`, the dataset configuration file should include the following attributes:

Structure and Loading

- `path`: Directory path containing the `features_subdir` directory and corresponding CSV files (such as `train.csv`, `dev.csv`, and `test.csv`).
- `features_subdir`: Optional subdirectory within the dataset path where (extracted) features are stored.
  - If not present and no preprocessing is used (e.g., for raw audio), the subdirectory defaults to `audio_subdir`.
  - For preprocessing transforms (e.g., log-Mel spectrograms with `log_mel_16k`), it should match the transform's name, and the processed features are saved in this subdirectory after preprocessing.
  - If features are stored outside of the dataset root directory (`path`), an absolute path can be specified to override the default directory structure outlined below.
- `features_path`: Optional root directory for the `features_subdir` directory, replacing the `path` attribute for the features. Useful when the features are stored in (or should be extracted to) a different directory than the root of the dataset. If not specified, the `path` attribute is used as the root directory for the features.
- `index_column`: Column in the CSV files containing the file paths, relative to the `features_subdir` directory.
- `target_column`: Column in the CSV files containing the corresponding targets or labels for each file.
- `file_type`: Specifies the type of files to be loaded (e.g., `wav`, `npy`, etc.).
- `file_handler`: The file handler used for loading the files.

This results in a directory structure like the following:

```
{path}/{features_subdir}/optional/subdirs/some.file
```

For instance, a file in the `index_column` might be `optional/subdirs/some.file`, where `some.file` is an audio or a feature file.

In order to load custom dataset splits that do not follow the standard `train.csv`, `dev.csv`, and `test.csv` convention, the `df_train`, `df_dev`, and `df_test`, properties of the dataset class can be overwritten (see custom datasets tutorial).

Training and Evaluation

- `criterion`: The criterion to use for training.
- `metrics`: A list of metrics to evaluate the model.
- `tracking_metric`: The metric to track for early stopping and model selection.
- `transform`: The online transforms to apply to the data and the output `type` of the dataset.
- `train_loader_kwargs`, `dev_loader_kwargs`, and `test_loader_kwargs`: Additional keyword arguments for the `DataLoader` such as the `num_workers`, `prefetch_factor`, etc. The keyword arguments can also be specified globally in

the main configuration file, which will be passed to all datasets. However, the dataset-specific keyword arguments will overwrite the global ones.

> **Note**
>
> The following attributes are automatically passed to the dataset during initialization and determined at runtime:
>
> - `train_transform`, `dev_transform`, and `test_transform`: The `SmartCompose` transformation pipelines (which may include possible online transforms or augmentations).
> - `seed`: The random seed for reproducibility during training.
>
> The `transform` attribute in the configuration is not passed to the dataset during initialization and is used to specify the type of data the dataset provides as well as any online transforms to be applied to the data at runtime.

To avoid race conditions when using Launcher Plugins that may run multiple training jobs in parallel, autrainer fetch and autrainer preprocess or `fetch()` and `preprocess()` are used to download the dataset and preprocess the data before training.

> **Note**
>
> All datasets that are provided by *autrainer* can be automatically downloaded as well as optionally preprocessed using the autrainer fetch and autrainer preprocess CLI commands or the `fetch()` and `preprocess()` CLI wrapper functions.

All *autrainer* datasets and loaders return DataItem and DataBatch structs which represent the data items and batches of data, respectively.

Abstract Dataset

All datasets inherit from the `AbstractDataset` class.

Base Datasets

Base datasets that can be used for training without the need for creating custom datasets.

Toy Datasets

A toy dataset for testing purposes.

> **Note**
>
> To easily test implementations, multiple toy dataset configurations across modalities and tasks are provided. We offer `ToyAudio-...` for audio, `ToyImage-...` for image, and `ToyTabular-...` for tabular data, respectively. For each dataset, we provide a task `-R` for regression, `-C` for classification, `-MLC` for multi-label classification, and `-MTR` for multi-target regression.

Audio Datasets

We provide a number of different audio-specific datasets.

Dataset Utils

Dataset utilities provide file handlers, target (or label) transforms, and a dataset wrapper for datasets.

File Handlers

File handlers are used to load and save files and are specified using shorthand syntax in the dataset and preprocessing configurations.

> **Tip**
>
> To create custom file handlers, refer to the custom file handlers tutorial.

Target Transforms

Target transforms are specified using shorthand syntax in the dataset configuration and used to encode as well as decode the targets (or labels) of the dataset. Additionally, target transforms provide functiuons for batch prediction and majority voting.

> **Tip**
>
> To create custom target transforms, refer to the custom target transforms tutorial.

Dataset Wrapper

The `DatasetWrapper` provides a wrapper around a `torch.utils.data.Dataset`, utilizing the file handlers and target transforms to load and transform the data, returning the data, target (or label), and index of each sample.

ZIP Downloader

To automatically download and extract zip files, the `ZipDownloadManager` can be used.

Loggers

Loggers can be added by specifying a list of `loggers` in the main configuration using Shorthand Syntax.

> **Tip**
>
> To create custom loggers, refer to the custom loggers tutorial.

For example, to add a `MLFlowLogger`, specify it in the main configuration by adding a list of `loggers`:

conf/config.yaml

```yaml
...
loggers:
  - autrainer.loggers.MLFlowLogger:
      output_dir: ${results_dir}/.mlflowruns
...
```

Abstract Logger

All loggers inherit from the `AbstractLogger` class.

class autrainer.loggers.AbstractLogger(*exp_name*, *run_name*, *metrics*, *tracking_metric*, *artifacts=['model_summary.txt', 'metrics.csv', {'config.yaml': '.hydra'}]*)

Base class for loggers.

Parameters:

- **exp_name** (`str`) – The name of the experiment.
- **run_name** (`str`) – The name of the run.
- **metrics** (`List`[`AbstractMetric`]) – The metrics to log.
- **tracking_metric** (`AbstractMetric`) – The metric to determine the best results.
- **artifacts** (`List`[`Union`[`str`, `Dict`[`str`, `str`]]]) – The artifacts to log. Defaults to `ARTIFACTS`.

log_and_update_metrics(*metrics*, *iteration=None*)

Log all metrics in the metrics dictionary for the given iteration. Automatically updates the best metrics based on the tracking metric.

Parameters:

- **metrics** (`Dict`[`str`, `Union`[`int`, `float`]]) – The metrics to log, with the metric name as the key and the metric value as the value.
- **iteration** (`int`) – The iteration, epoch, or step number. If None, the iteration will not be logged (e.g., for test metrics). Defaults to None.

Return type:

`None`

setup()

Optional setup method called at the beginning of the run (*cb_on_train_begin*).

Return type:

`None`

abstractmethod log_params(*params*)

Log the parameters of the configuration.

Parameters:

params (`Union`[`dict`, `DictConfig`]) – The parameters of the configuration.

Return type:

`None`

abstractmethod log_metrics(*metrics*, *iteration=None*)

Log the metrics for the given iteration.

Parameters:

- **metrics** (`Dict`[`str`, `Union`[`int`, `float`]]) – The metrics to log, with the metric name as the key and the metric value as the value.
- **iteration** – The iteration, epoch, or step number. If None, the iteration will not be logged (e.g., for test metrics). Defaults to None.

Return type:

None

abstractmethod log_timers(*timers*)

Log all timers (e.g., mean times for train, dev, and test).

Parameters:

timers (Dict[str, float]) – The timers to log, with the timer name as the key and the timer value as the value.

Return type:

None

abstractmethod log_artifact(*filename*, *path=''*)

Log an artifact (e.g., a file).

Parameters:

- **filename** (str) – The name of the artifact.
- **path** (str) – The absolute path or relative path from the current working directory to the artifact. Defaults to "".

Return type:

None

end_run()

Optional end run method called at the end of the run (*cb_on_train_end*).

Return type:

None

Optional Loggers

The MLFlowLogger logs data to MLFlow, while the TensorBoardLogger logs data to TensorBoard. Both loggers require additional dependencies, which are not installed by default. To install all necessary dependencies, refer to the Installation section.

To start the MLflow or TensorBoard server, run the following commands:

```
mlflow server --backend-store-uri /path/to/results/.mlflowruns
tensorboard --logdir /path/to/results/.tensorboard
```

In both cases, the path should be the same as the `output_dir` specified in the configuration.

class autrainer.loggers.MLFlowLogger(*exp_name*, *run_name*, *metrics*, *tracking_metric*, *artifacts=['model_summary.txt', 'metrics.csv', {'config.yaml': '.hydra'}]*, *output_dir='mlruns'*)

Base class for loggers.

Parameters:

- **exp_name** (str) – The name of the experiment.
- **run_name** (str) – The name of the run.

- **metrics** (`List[AbstractMetric]`) – The metrics to log.
- **tracking_metric** (`AbstractMetric`) – The metric to determine the best results.
- **artifacts** (`List[Union[str, Dict[str, str]]]`) – The artifacts to log. Defaults to `ARTIFACTS`.

class **autrainer.loggers.TensorBoardLogger**(***exp_name***, ***run_name***, ***metrics***, ***tracking_metric***, ***artifacts***=*['model_summary.txt', 'metrics.csv', {'config.yaml': '.hydra'}]*, ***output_dir***='*runs*')

Base class for loggers.

Parameters:

- **exp_name** (`str`) – The name of the experiment.
- **run_name** (`str`) – The name of the run.
- **metrics** (`List[AbstractMetric]`) – The metrics to log.
- **tracking_metric** (`AbstractMetric`) – The metric to determine the best results.
- **artifacts** (`List[Union[str, Dict[str, str]]]`) – The artifacts to log. Defaults to `ARTIFACTS`.

Fallback Logger

If a logger such as `MLFlowLogger` is specified in the configuration, but the required dependencies are not installed, the `FallbackLogger` will be used instead. This logger will log a warning message and will not log any data.

class **autrainer.loggers.FallbackLogger**(***requested_logger***=*None*, ***extras***=*None*)

Fallback logger for when a requested logger is not available.

If the requested logger is not available, a warning is logged. If both requested_logger and extras are None, nothing is logged. The logger serves as a no-op.

Parameters:

- **requested_logger** (`Optional[str]`) – The requested logger. Defaults to None.
- **extras** (`Optional[str]`) – The extras required to install the logger. Defaults to None.

Metrics

Metrics are specified in the dataset configuration using shorthand syntax. The `tracking_metric` attribute specifies the metric to be used for early stopping. The `metrics` attribute in the dataset configuration specifies a list of metrics to be used during training.

> **Tip**
>
> To create custom metrics, refer to the custom metrics tutorial.

For example, a dataset for classification could specify the following metrics:

conf/dataset/ExampleDataset.yaml

```yaml
id: ExampleDataset
_target_: example_dataset.ExampleDataset
...

tracking_metric: autrainer.metrics.Accuracy
metrics:
  - autrainer.metrics.Accuracy
  - autrainer.metrics.UAR
```

```
- autrainer.metrics.F1
```

Abstract Metric

class **autrainer.metrics.AbstractMetric**(*name*, *fn*, *fallback*, ***fn_kwargs*)

Abstract class for metrics.

Parameters:

- **name** (`str`) – The name of the metric.
- **fn** (`Callable`) – The function to compute the metric.
- **fallback** (`float`) – The fallback value if the metric is NaN.
- ****fn_kwargs** (`dict`) – Additional keyword arguments to pass to the function.

__call__(**args*, ***kwargs*)

Compute the metric.

Parameters:

- ***args** – Positional arguments for the function.
- ****kwargs** – Keyword arguments for the function.

Return type:

`float`

Returns:

The score.

abstract property **starting_metric**: *float*

The starting metric value.

Returns:

The starting metric value.

abstract property **suffix**: *str*

The suffix of the metric.

Returns:

The suffix of the metric.

abstractmethod static **get_best**(*a*)

Get the best metric value from a series of scores.

Parameters:

a (`Union`[`Series`, `ndarray`]) – Pandas series or numpy array of scores.

Return type:

`float`

Returns:

Best metric value.

abstractmethod static **get_best_pos**(*a*)

Get the position of the best metric value from a series of scores.

Parameters:

a (`Union`[`Series`, `ndarray`]) – Pandas series or numpy array of scores.

Return type:

`int`

Returns:

Position of the best metric value.

abstractmethod static **compare**(*a*, *b*)

Compare two scores and return True if the first score is better.

Parameters:

- **a** (`Union`[`int`, `float`]) – First score.
- **b** (`Union`[`int`, `float`]) – Second score.

Return type:

`bool`

Returns:

True if the first score is better.

unitary(*y_true*, *y_pred*)

Unitary evaluation of metric.

Metric computed for each individual label in the case of multilabel classification or regression.

In the default case, computes the metric in the same way as globally.

Parameters:

- **y_true** (`ndarray`) – ground truth values.
- **y_pred** (`ndarray`) – prediction values.

Return type:

`float`

Returns:

The unitary score.

class autrainer.metrics.**BaseAscendingMetric**(*name*, *fn*, *fallback=None*, ***fn_kwargs*)

Base for ascending metrics with higher values being better.

Parameters:

- **name** (`str`) – The name of the metric.
- **fn** (`Callable`) – The function to compute the metric.
- **fallback** (`float`) – The fallback value if the metric is NaN. If None, the fallback value is set to -1e32. Defaults to None.
- ****fn_kwargs** (`dict`) – Additional keyword arguments to pass to the function.

property **starting_metric**: *float*

Ascending metric starting value.

Returns:

-1e32

property **suffix**: *str*

Ascending metric suffix.

Returns:

“max”

class **autrainer.metrics.BaseDescendingMetric**(*name*, *fn*, *fallback=None*, ***fn_kwargs*)

Base for descending metrics with lower values being better.

Parameters:

- **name** (`str`) – The name of the metric.
- **fn** (`Callable`) – The function to compute the metric.
- **fallback** (`float`) – The fallback value if the metric is NaN. If None, the fallback value is set to 1e32. Defaults to None.
- ****fn_kwargs** (`dict`) – Additional keyword arguments to pass to the function.

property **starting_metric**: *float*

Descending metric starting value.

Returns:

1e32

property **suffix**: *str*

Descending metric suffix.

Returns:

“min”

Classification Metrics

class **autrainer.metrics.Accuracy**

Accuracy metric using *audmetric.accuracy*.

class **autrainer.metrics.UAR**

Unweighted average recall metric using *audmetric.unweighted_average_recall*.

class **autrainer.metrics.F1**

F1 metric using *audmetric.unweighted_average_fscore*.

Multi-label Classification Metrics

class **autrainer.metrics.MLAccuracy**

Accuracy metric using *sklearn.metrics.accuracy_score*.

class **autrainer.metrics.MLF1Macro**

F1 macro metric using *sklearn.metrics.f1_score*.

class **autrainer.metrics.MLF1Micro**

F1 micro metric using *sklearn.metrics.f1_score*.

class **autrainer.metrics.MLF1Weighted**

F1 weighted metric using *sklearn.metrics.f1_score*.

(Multi-target) Regression Metrics

class **autrainer.metrics.CCC**

Concordance correlation coefficient metric using *audmetric.concordance_cc* for (multi-target) regression. The metric is calculated for each target separately and the mean is returned.

class **autrainer.metrics.MAE**

Mean absolute error metric using *audmetric.mean_absolute_error* for (multi-target) regression. The metric is calculated for each target separately and the mean is returned.

class **autrainer.metrics.MSE**

Mean squared error metric using *audmetric.mean_squared_error* for (multi-target) regression. The metric is calculated for each target separately and the mean is returned.

class **autrainer.metrics.PCC**

Pearson correlation coefficient metric using *audmetric.pearson_cc* for (multi-target) regression. The metric is calculated for each target separately and the mean is returned.

Models

autrainer provides a number of different audio-specific models as well as wrappers for torchvision and timm models.

> **Tip**
>
> To create custom models, refer to the custom models tutorial.

Default configurations that end in `-T` indicate that the model uses transfer learning with pretrained weights. To avoid race conditions when using Launcher Plugins that may run multiple training jobs in parallel, autrainer fetch or `fetch()` are used to download the pretrained weights before training.

> **Note**
>
> Most models are pretrained on the ImageNet or AudioSet datasets. To ensure compatibility with any number of output dimensions, the last linear layer of the model is replaced with a new linear layer with the correct number of output

dimensions and will therefore **not** be pretrained.

The weights for all pretrained models that are provided by *autrainer* can be automatically downloaded using the autrainer fetch CLI command or the `fetch()` CLI wrapper function.

To optionally use model, optimizer, or scheduler checkpoints, the following attributes can be set in any model configuration file:

- `model_checkpoint`: The path to the model checkpoint file. Defaults to None.
- `optimizer_checkpoint`: The path to the optimizer checkpoint file. Defaults to None.
- `scheduler_checkpoint`: The path to the scheduler checkpoint file. Defaults to None.
- `skip_last_layer`: Whether to skip loading the state of the last linear or convolutional layer. When set to True, the state of the last layer (if present) is omitted from both the model and optimizer, allowing for training on a different target dataset with varying output dimensions. Defaults to True.

Note

Loading a checkpoint assumes that the model architecture is the same as the one used to create the checkpoint and that the last layer of the model is specified as the final `Linear` or `_ConvNd` module. If the last layer is not the final layer in the module order, it may not be correctly identified for skipping.

Abstract Model

All models inherit from the `AbstractModel` class and implement the `forward()` and `embeddings()` methods. These models must adhere a specific signature for their `forward()` method as *autrainer* uses argument names for its internal data flow. For this reason, `features` must always be the name of the first argument. All subsequent arguments can be named differently but they must be present as attributes in the corresponding `AbstractDataItem`.

Model Wrappers

For convenience, we provide wrappers for torchvision and timm models.

Audio Models

autrainer provides a number of different audio-specific models.

Optimizers

Any torch optimizer or custom optimizer can be used for training.

Tip

To create custom optimizers, refer to the custom optimizers tutorial.

Torch Optimizers

Torch optimizers (`torch.optim`) can be specified as relative python import paths for the `_target_` attribute in the configuration file. Any additional attributes (except `id`) are passed as keyword arguments to the optimizer constructor.

For example, `torch.optim.Adam` can be used as follows:

conf/optimizer/Adam.yaml

```yaml
id: Adam
_target_: torch.optim.Adam
```

autrainer provides a number of default configurations for torch optimizers:

Default Configurations

conf/optimizer/SGD.yaml

```yaml
id: SGD
_target_: torch.optim.SGD
```

conf/optimizer/Adam.yaml

```yaml
id: Adam
_target_: torch.optim.Adam
```

conf/optimizer/AdamW.yaml

```yaml
id: AdamW
_target_: torch.optim.AdamW
```

conf/optimizer/Adadelta.yaml

```yaml
id: Adadelta
_target_: torch.optim.Adadelta
```

conf/optimizer/Adagrad.yaml

```yaml
id: Adagrad
_target_: torch.optim.Adagrad
```

conf/optimizer/Adamax.yaml

```yaml
id: Adamax
_target_: torch.optim.Adamax
```

conf/optimizer/NAdam.yaml

```yaml
id: NAdam
_target_: torch.optim.NAdam
```

conf/optimizer/RAdam.yaml

```yaml
id: RAdam
_target_: torch.optim.RAdam
```

conf/optimizer/SparseAdam.yaml

```yaml
id: SparseAdam
_target_: torch.optim.SparseAdam
```

conf/optimizer/RMSprop.yaml

```yaml
id: RMSprop
_target_: torch.optim.RMSprop
```

conf/optimizer/Rprop.yaml

```yaml
id: Rprop
_target_: torch.optim.Rprop
```

conf/optimizer/ASGD.yaml

```yaml
id: ASGD
_target_: torch.optim.ASGD
```

conf/optimizer/LBFGS.yaml

```yaml
id: LBFGS
_target_: torch.optim.LBFGS
```

Custom Optimizers

Custom Step Function

Custom optimizers can optionally provide a `custom_step()` function that is called instead of the standard training step and should be defined as follows:

custom_step function of an optimizer

```python
class SomeOptimizer(torch.optim.Optimizer):
    def custom_step(
        self,
        model: AbstractModel,
        data: DataBatch,
        criterion: torch.nn.Module,
        probabilities_fn: Callable,
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """Custom step function for the optimizer.

        Args:
            model: The model to train.
            data: The data batch containing features, target, and potentially
                additional fields. The data batch is on the same
                device as the model. Additional fields are passed to the model
                as keyword arguments if they are present in the model's forward
                method.
            criterion: Loss function.
            probabilities_fn: Function to convert model outputs to
                probabilities.

        Returns:
            Tuple containing the non-reduced loss and model outputs.
        """
```

Note

The `custom_step()` function should perform both the forward and backward pass as well as update the model parameters accordingly.

Postprocessing

Postprocessing allows for the summarization, aggregation, as well as grouping of the results of the grid search and can be done using the postprocessing CLI commands or the postprocessing CLI wrapper functions.

Summarization

`SummarizeGrid` is used to summarize the results of the grid search. For each metric, a plot is created. All validation and test results are stored in a DataFrame. In addition, a DataFrame summarizing the hyperparameters is created.

class **autrainer.postprocessing.SummarizeGrid(*results_dir*, *experiment_id*, *summary_dir='summary'*, *training_dir='training'*, *clear_old_outputs=True*, *training_type=None*, *max_runs_plot=None*, *plot_params=None*)**

Summarize the results of a grid search.

Parameters:

- **results_dir** (`str`) – The directory where the results are stored.
- **experiment_id** (`str`) – The ID of the grid search experiment.
- **summary_dir** (`str`) – The directory where the the grid search summary will be stored. Defaults to “summary”.
- **training_dir** (`str`) – The directory of the training results of the experiment. Defaults to “training”.
- **clear_old_outputs** (`bool`) – Whether to clear existing summary outputs. Defaults to True.
- **training_type** (`Optional`[`str`]) – The type of training in [“Epoch”, “Step”]. If None, it will be inferred from the training results. Defaults to None.
- **max_runs_plot** (`Optional`[`int`]) – The maximum number of best runs to plot. If None, all runs will be plotted. Defaults to None.
- **plot_params** (`Union`[`DictConfig`, `Dict`, `None`]) – Additional parameters for plotting. Defaults to None.

summarize()

Summarize the results of the grid search.

Return type:

`None`

plot_metrics()

Plot the metrics of the grid search.

Return type:

`None`

plot_aggregated_bars()

Plot additional aggregated bars for the grid search.

Return type:

`None`

Aggregation

`AggregateGrid` is used to aggregate the results of the grid search. The results are aggregated over one or more hyperparameters.

class **autrainer.postprocessing.AggregateGrid(*results_dir*, *experiment_id*, *aggregate_list*, *aggregate_prefix='agg'*, *training_dir='training'*, *max_runs_plot=None*, *aggregate_name=None*, *aggregated_dict=None*, *plot_params=None*)**

Aggregate the results of a grid search over one or more parameters.

If loggers have been used for the grid search, the aggregated results will be logged to the same loggers.

Parameters:

- **results_dir** (`str`) – The directory where the results are stored.
- **experiment_id** (`str`) – The ID of the grid search experiment.
- **aggregate_list** (`List[str]`) – The list of parameters to aggregate over.
- **aggregate_prefix** (`str`) – The prefix for the aggregated experiment ID. Defaults to “agg”.
- **training_dir** (`str`) – The directory of the training results of the experiment. Defaults to “training”.
- **max_runs_plot** (`Optional[int]`) – The maximum number of best runs to plot. If None, all runs will be plotted. Defaults to None.
- **aggregate_name** (`Optional[str]`) – The name of the aggregated experiment. If None, it will be generated from the aggregate_list. Defaults to None.
- **aggregated_dict** (`Optional[dict]`) – A dictionary mapping the aggregated experiment names to the runs to aggregate. If None, the runs will be aggregated based on the aggregate_list. Defaults to None.
- **plot_params** (`Optional[dict]`) – Additional parameters for plotting. Defaults to None.

aggregate()

Aggregate the runs based on the specified parameters.

Return type:

`None`

summarize()

Summarize the aggregated results.

Return type:

`None`

Grouping

`GroupGrid` is used to manually group the results of the grid search using a Hydra configuration file. A configuration file is used to define the groups which can be any combination of runs. The results are grouped according to the configuration file and can span multiple experiments.

The following configuration file illustrates the structure of the configuration file:

Manual Grouping Example

Manual grouping is done by defining a YAML configuration file as shown below. Multiple experiments (*exp1*, *exp2*, ...) can be created hosting the grouped runs. `runs` is a list of runs that are created for each experiment.

conf/grouping.yaml

```yaml
defaults:
  - _hydra_disable_logging_
  - _self_
  - plotting: Default # Use the default plotting configuration

results_dir: results # Directory to save results
max_runs: null # Maximum number of best runs to include in the summary plots

groupings:
  - experiment_id: exp1 # Experiment ID (will be created if it doesn't exist)
    create_summary: true # Whether to create a summary for the experiment
    dir: null # Optional global directory for all runs
    id: null # Optional global ID for all runs
    states: null # Optional global save states for all runs
    runs:
      - run_name: FirstRun # Run name
        dir: some_results_dir # Directory for the runs to be grouped
        id: some_exp # ID for the runs to be grouped
        states: false # Whether to copy the model states
        combine: # Runs to combine into run_name
          - SomeRun1
          - SomeRun2
      - run_name: SecondRun
        dir: some_results_dir
        id: some_exp
        states: false
        combine:
          - SomeRun3
          - SomeRun4
  - experiment_id: exp2 # Example with global parameters to be more concise
    create_summary: true
    dir: some_results_dir
    id: some_exp
    states: false
    runs:
      - run_name: FirstRun
        combine:
          - SomeRun1
          - SomeRun2
      - run_name: SecondRun
        combine:
          - SomeRun3
          - SomeRun4
```

class autrainer.postprocessing.GroupGrid(*results_dir*, *groupings*, *max_runs=None*, *plot_params=None*)

Group runs of one or more grid search experiments based on the specified groupings.

Parameters:

- **results_dir** (`str`) – The directory where the results are stored.
- **groupings** (`Union`[`DictConfig`, `List`[`Dict`]]) – A list of experiments to create containing one or more runs to group.
- **max_runs_plot** – The maximum number of best runs to plot. If None, all runs will be plotted. Defaults to None.
- **plot_params** (`Optional`[`dict`]) – Additional parameters for plotting. Defaults to None.

group_runs()

Group the runs of the specified experiments based on the groupings.

Return type:

`None`

Schedulers

Schedulers are optional and by default not used. This is indicated by the absence of the `scheduler` attribute in the sweeper configuration (implicitly set to a `None` configuration file).

In addition to the `id`, `_target_`, and scheduler-specific attributes, schedulers can have the following attributes:

- `step_frequency` (str): The frequency at which the step function is called during training. Possible values are:
  - `batch`: The step function is called after every batch. Defaults to `batch` if not specified.
  - `evaluation`: The step function is called after the number of iterations specified by the `eval_frequency` of the training configuration.

To use a scheduler, specify it in the configuration file (`conf/config.yaml`) for the sweeper.

> **Tip**
>
> To create custom schedulers, refer to the custom schedulers tutorial.

Torch Schedulers

Any scheduler from `torch.optim.lr_scheduler` can be used. A scheduler is specified in the configuration file as a relative import path for the `_target_` argument. Any additional arguments (except the `id` and `_target_`) are passed as keyword arguments to the scheduler constructor.

For example, the `torch.optim.lr_scheduler.StepLR` scheduler can be used as follows:

conf/scheduler/StepLR.yaml

```yaml
id: StepLR
_target_: torch.optim.lr_scheduler.StepLR
step_size: 30

step_frequency: evaluation
```

Default Configurations

None

This configuration file is used to indicate that no scheduler is used and serves as a no-op placeholder.

conf/scheduler/None.yaml

```yaml
id: None
_target_: None
```

StepLR

conf/scheduler/StepLR.yaml

```yaml
id: StepLR
_target_: torch.optim.lr_scheduler.StepLR
step_size: 30

step_frequency: evaluation
```

Inference

Inference offers an interface to obtain predictions or embeddings from a trained model. In addition, a sliding window can be used to obtain predictions or embeddings from parts of the input data.

The autrainer inference CLI command and the `inference()` CLI wrapper function allow for the (sliding window) inference of audio data using a trained model.

> **Note**
>
> Currently, inference is only supported for audio data.

Audio Inference

Training

autrainer supports both `epoch`- and `step`-based training.

> **Tip**
>
> To create custom training configurations, refer to the custom training configurations quickstart.

Configuring Training

Training is configured in the main configuration file and comprises the following attributes:

- `iterations`: The number of iterations to train the model for.
- `training_type`: The type of training, either `epoch` or `step`. By default, it is set to `epoch`.
- `eval_frequency`: The frequency in terms of iterations to evaluate the model on the development set. By default, it is set to 1.
- `save_frequency`: The frequency in terms of iterations to states of the model, optimizer, and scheduler. By default, it is set to 1.
- `inference_batch_size`: The batch size to use during inference. By default, it is set to the training batch size.

The following optional attributes can be set to configure the training process:

- `progress_bar`: Whether to display a progress bar during training and evaluation. By default, it is set to True.
- `continue_training`: Whether to continue from an already finished run with the same configuration and fewer iterations. By default, it is set to True.
- `remove_continued_runs`: Whether to remove the runs that have been continued. By default, it is set to True.
- `save_train_outputs`: Whether to save indices, targets, losses, outputs, and predictions (results) on the training set. By default, it is set to True.
- `save_dev_outputs`: Whether to save indices, targets, losses, outputs, and predictions (results) on the development set. By default, it is set to True.
- `save_test_outputs`: Whether to save indices, targets, losses, outputs, and predictions (results) on the test set. By default, it is set to True.

For brevity, all training attributes with default values are outsourced to the _autrainer_.yaml defaults file and imported in the main configuration file.

> **Note**

> Throughout the documentation, the term *iteration* (as well as `iterations`, `eval_frequency`, and `save_frequency`) refers to a full pass over the training set for epoch-based training, and a single optimization step over a batch of the training set for step-based training.

Trainer

`ModularTaskTrainer` manages the training process. It instantiates the model, dataset, criterion, optimizer, scheduler, and callbacks, and trains the model on the dataset. It also logs the training process and saves the model, optimizer, and scheduler states at the end of each epoch.

The `cfg` of the trainer is the composed main configuration file (e.g., `conf/config.yaml`) for each training configuration in the sweep.

Callbacks

Any dataset, model, optimizer, scheduler, criterion, or logger can specify callback functions which start with `cb_on_*()`.

Each callback is automatically invoked at the appropriate time during training. The function signature of each callback is defined in `CallbackSignature`.

For more control over the training process, custom callbacks can be defined and added to the trainer by specifying a list of callback classes using shorthand syntax in the `callbacks` attribute of the main configuration file. Each callback class can specify any number of callback functions following the signatures defined in `CallbackSignature`.

> **Tip**
>
> To create custom callbacks, refer to the custom callbacks tutorial.

Transforms

Transforms serve the purpose of preprocessing the input data before it is fed to the model and are specified as pipelines with Shorthand Syntax. This can be done both offline (preprocessing) and online.

> **Tip**
>
> To create custom transforms, refer to the custom transforms tutorial.

Multiple transforms are combined into a pipeline, which is handled by the `TransformManager`. Pipelines are automatically assembled and sorted based on the `order` attribute of each transform, which is handled by the `SmartCompose`.

Transforms with a lower order are applied first, followed by transforms with a higher order. If two transforms share the same `order`, they are applied in the order they are specified in the configuration.

Both types of transforms can utilize any of the available transforms provided by *autrainer*, or custom transforms inheriting from the `AbstractTransform` class.

While the choice to use offline or online transforms depends on the use case and is a tradeoff between storage and computational costs, both can be used in conjunction for maximum flexibility.

> **Note**
>
> Some transforms (such as `FeatureExtractor`) may require additional resources (e.g., tokenizers), which can be automatically downloaded using the autrainer fetch CLI command or the `fetch()` CLI wrapper function after

specifying the transform in the model or dataset configuration.

Preprocessing Transforms

Preprocessing transforms are specified in the dataset configuration aligning the `features_subdir` with the name of the preprocessing file. This way, the processed data is stored in a subdirectory of the dataset directory with the same name as the preprocessing file, or in a different path altogether by specifying the `features_path` attribute in the configuration file. To save the processed data, the `file_handler` specified in the dataset configuration is used.

> **Tip**
>
> To create custom preprocessing transforms, refer to the custom preprocessing transforms tutorial.

Preprocessing transforms consist of the following attributes:

- `file_handler`: The file handler to use for loading the data specified with Shorthand Syntax. To discover all available file handlers, refer to the file handlers section.
- `pipeline`: The sequence of transformations to apply to the data. The pipeline is specified using Shorthand Syntax and can include any of the available transforms.

> **Note**
>
> To avoid race conditions during parallel training, the preprocessing is applied to the entire dataset before training.
>
> To automatically preprocess all datasets specified in the main configuration (`conf/config.yaml`) file, the following preprocessing CLI command can be used:
>
> ```
> autrainer preprocess
> ```
>
> Alternatively, use the following preprocessing CLI wrapper function:
>
> ```
> autrainer.cli.preprocess()
> ```

> **Warning**
>
> Following *v0.6.0*, *autrainer* iterates over all files in all datasets and extracts the respective features without replacement. It is possible that features have been extracted for a subset of the dataset. This may happen when different dataset subsets are supported. Therefore, it is recommended that *autrainer preprocess* is always called before training. Features have to be manually deleted to overwrite.

autrainer offers default configurations for log-Mel spectrogram extraction and openSMILE feature extraction.

Default Configurations

Log-Mel Spectrograms

Log-Mel spectrograms are a common representation of audio data.

autrainer offers default configurations for log-Mel spectrogram extraction for different sampling rates. The generation process is adapted from: PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition.

conf/preprocessing/log_mel_16k.yaml

```yaml
file_handler:
  autrainer.datasets.utils.AudioFileHandler:
    target_sample_rate: 16000
pipeline:
  - autrainer.transforms.StereoToMono
  - autrainer.transforms.PannMel:
      sample_rate: 16000
      window_size: 512
      hop_size: 160
      mel_bins: 64
      fmin: 50
      fmax: 8000
      ref: 1.0
      amin: 1e-10
      top_db: null

# adapted from:
# https://github.com/qiuqiangkong/audioset_tagging_cnn/blob/d2f4b8c18eab44737fcc0de1248ae21eb43f6aa4/
```

conf/preprocessing/log_mel_32k.yaml

```yaml
file_handler:
  autrainer.datasets.utils.AudioFileHandler:
    target_sample_rate: 32000
pipeline:
  - autrainer.transforms.StereoToMono
  - autrainer.transforms.PannMel:
      sample_rate: 32000
      window_size: 1024
      hop_size: 320
      mel_bins: 64
      fmin: 50
      fmax: 14000
      ref: 1.0
      amin: 1e-10
      top_db: null

# adapted from:
# https://github.com/qiuqiangkong/audioset_tagging_cnn/blob/d2f4b8c18eab44737fcc0de1248ae21eb43f6aa4/
```

conf/preprocessing/log_mel_8k.yaml

```yaml
file_handler:
  autrainer.datasets.utils.AudioFileHandler:
    target_sample_rate: 8000
pipeline:
  - autrainer.transforms.StereoToMono
  - autrainer.transforms.PannMel:
      sample_rate: 8000
      window_size: 256
      hop_size: 80
      mel_bins: 64
      fmin: 50
      fmax: 4000
      ref: 1.0
      amin: 1e-10
      top_db: null

# adapted from:
# https://github.com/qiuqiangkong/audioset_tagging_cnn/blob/d2f4b8c18eab44737fcc0de1248ae21eb43f6aa4/
```

openSMILE Features

openSMILE is a widely used feature extraction tool for audio data.

autrainer provides default configurations for extracting openSMILE features. openSMILE is an optional dependency and may need to be installed separately. For installation, refer to the installation guide.

conf/preprocessing/eGeMAPSv02-llds.yaml

```yaml
file_handler:
  autrainer.datasets.utils.AudioFileHandler:
    target_sample_rate: 16000
pipeline:
  - autrainer.transforms.StereoToMono
  - autrainer.transforms.OpenSMILE:
      feature_set: eGeMAPSv02
      sample_rate: 16000
```

conf/preprocessing/eGeMAPSv02.yaml

```yaml
file_handler:
  autrainer.datasets.utils.AudioFileHandler:
    target_sample_rate: 16000
pipeline:
  - autrainer.transforms.StereoToMono
  - autrainer.transforms.OpenSMILE:
      feature_set: eGeMAPSv02
      sample_rate: 16000
      functionals: true
```

conf/preprocessing/ComParE_2016-llds.yaml

```yaml
file_handler:
  autrainer.datasets.utils.AudioFileHandler:
    target_sample_rate: 16000
pipeline:
  - autrainer.transforms.StereoToMono
  - autrainer.transforms.OpenSMILE:
      feature_set: ComParE_2016
      sample_rate: 16000
```

conf/preprocessing/ComParE_2016.yaml

```yaml
file_handler:
  autrainer.datasets.utils.AudioFileHandler:
    target_sample_rate: 16000
pipeline:
  - autrainer.transforms.StereoToMono
  - autrainer.transforms.OpenSMILE:
      feature_set: ComParE_2016
      sample_rate: 16000
      functionals: true
```

Online Transforms

Online transforms can be specified with the `transform` attribute in both the model and dataset, applied to the input data before it is passed to the model. Each pipeline is specified as a list of transforms using Shorthand Syntax.

Model and dataset configurations specify the transforms to be applied to the input data. If augmentations are used, they are merged with the online transforms to create a single pipeline.

> **Tip**
>
> To create custom online transforms, refer to the custom online transforms tutorial.

Types

Each `transform` configuration includes a `type` attribute, specifying the type of data the model expects or the dataset provides:

- `image`: RGB images
- `grayscale`: Grayscale images (e.g., spectrograms or other single-channel images)
- `raw`: Raw audio waveforms
- `tabular`: Tabular data (e.g., openSMILE features)

> **Note**
>
> The conversion between RGB and grayscale images is handled automatically by the pipeline to ensure compatibility between models and datasets.

Pipelines

Different transforms can be specified for `train`, `dev`, and `test` pipelines. In addition, `base` provides a common set of transforms to include in all pipelines.

For example, the following configuration applies different `RandomCrop` transforms for the `train`, `dev`, and `test` pipelines (which may not necessarily be useful in practice):

conf/model/SpectrogramModel.yaml

```yaml
...
transform:
  type: grayscale
  train:
    - autrainer.transforms.RandomCrop:
        size: 111
  dev:
    - autrainer.transforms.RandomCrop:
        size: 121
  test:
    - autrainer.transforms.RandomCrop:
        size: 131
```

Removing and Overriding

Models and datasets can remove any existing transform if it exists to ensure compatibility between datasets and models. Removal is done by setting the transform to `null` in the configuration. If a transform is set to `null` but is not present in the pipeline, it is ignored.

For example, a model taking in spectrograms could set the `Normalize` transform to `null` to remove it from the pipeline as it is not needed for spectrograms:

conf/model/SpectrogramModel.yaml

```yaml
...
transform:
  type: grayscale
  base:
    - autrainer.transforms.Normalize: null
```

Model online transforms outweigh dataset online transforms. If a model specifies a transform that is also specified in the dataset, the model transform is used, overriding the dataset transform.

> **Note**

Removing and overriding transforms is useful for ensuring compatibility between models and datasets, however, both are bound to the same pipeline (e.g. `base`, `train`, `dev`, or `test`).

If multiple transforms of the same type are specified in a pipeline, overriding or removing a transform affects all transforms of that type in the pipeline.

Tags

In case multiple transforms of the same type are specified in the pipeline, they can be tagged with a unique identifier using an `@` symbol followed by the tag name.

For example, the following configuration applies two `Normalize` transforms to the pipeline, each with a different tag:

conf/model/SpectrogramModel.yaml

```yaml
...
transform:
  type: grayscale
  base:
    - autrainer.transforms.Normalize@first:
        mean: 0.5
        std: 0.5
    - autrainer.transforms.Normalize@second:
        mean: 123
        std: 456
```

Transforms with tags can be removed or overridden analogously to transforms without tags, by appending the tag name to the transform name.

Note

If a transform with a tag is removed or overridden, only the transform with the specified tag (if it exists) is affected, allowing for the removal or overriding of specific transforms in the pipeline.

Transform Manager

The `TransformManager` is responsible for building the transformation pipeline based on the model and dataset configurations. In addition, it handles the inclusion of augmentations.

class **autrainer.transforms.TransformManager(*model_transform*, *dataset_transform*, *train_augmentation=None*, *dev_augmentation=None*, *test_augmentation=None*)**

Transform manager for composing transforms for the train, dev, and test datasets. Automatically handles the creation of all transformation pipelines including incorporating optional augmentations.

Parameters:

- **model_transform** (`Union`[`DictConfig`, `Dict`]) – The model transform configuration.
- **dataset_transform** (`Union`[`DictConfig`, `Dict`]) – The dataset transform configuration.
- **train_augmentation** (`Optional`[`SmartCompose`]) – The train augmentation pipeline. Defaults to None.
- **dev_augmentation** (`Optional`[`SmartCompose`]) – The dev augmentation pipeline. Defaults to None.
- **test_augmentation** (`Optional`[`SmartCompose`]) – The test augmentation pipeline. Defaults to None.

get_transforms()

Get the composed transform pipelines for the train, dev, and test datasets.

Return type:

`Tuple`[`SmartCompose`, `SmartCompose`, `SmartCompose`]

Returns:

The composed transform pipelines for the train, dev, and test

datasets.

Smart Compose

The `SmartCompose` is a helper that allows for the composition and ordering of transformations.

class autrainer.transforms.SmartCompose(*transforms*, ***kwargs*)

SmartCompose wrapper for torchvision.transforms.Compose, which allows for simple composition of transforms by adding them together. Transforms are automatically sorted by their order attribute if present.

Additionally, SmartCompose allows for the specification of a collate function to be used in a DataLoader, the last collate function specified will be used.

Parameters:

- **transforms** (`List`[`AbstractTransform`]) – List of transforms to compose.
- ****kwargs** – Additional keyword arguments to pass to torchvision.transforms.Compose.

__add__(*other*)

Add another transform to the composition. Transforms are automatically sorted by their order attribute if present.

Parameters:

other (`Union`[`SmartCompose`, `AbstractTransform`, `List`[`AbstractTransform`], `None`]) – Transform to add to the composition.

Raises:

TypeError – If the addition is not a valid type. Supported types are AbstractTransform, SmartCompose, and list of AbstractTransform or SmartCompose.

Return type:

`SmartCompose`

Returns:

New SmartCompose object with the added transform.

get_collate_fn(*data*)

Get the collate function. If no collate function is present in the transforms, the dataset default is returned. If multiple collate functions are present, the last one is used.

Parameters:

data (AbstractDataset) – Dataset to get the collate function for. Includes a `default_collate_fn`.

Return type:

`Callable`

Returns:

Collate function.

offset_generator_seed(*offset*)

Offset the generator seed for transforms that use random number generators. Useful for ensuring reproducibility and

randomness of augmentations when using multiple workers.

Parameters:

offset (`int`) – Offset to add to the generator seed. Usually the worker index.

Return type:

`None`

`__call__(item)`

Apply the transform to the input data item.

Parameters:

item (`AbstractDataItem`) – The input data item to transform.

Return type:

`AbstractDataItem`

Returns:

The transformed data item.

Abstract Transform

All transforms must inherit from `AbstractTransform` and implement the `__call__()` method.

class autrainer.transforms.AbstractTransform(*order=0*, ***kwargs*)

Abstract class for a transform.

Parameters:

- **order** (`int`) – The order of the transform in the pipeline. Larger means later in the pipeline. If multiple transforms have the same order, they are applied in the order they were added to the pipeline. Defaults to 0.
- **kwargs** – Additional keyword arguments to store in the object.

abstractmethod __call__(*item*)

Apply the transform to the input data item.

Parameters:

item (`AbstractDataItem`) – The input data item to transform.

Return type:

`AbstractDataItem`

Returns:

The transformed data item.

Available Transforms

autrainer provides a set of predefined transforms that can be used in any configuration.

class autrainer.transforms.AnyToTensor(*order=-100*)

Convert a numpy array, torch tensor, or a PIL image to a torch tensor.

Parameters:

order (`int`) – The order of the transform in the pipeline. Defaults to -100.

class autrainer.transforms.**Expand**(*size*, *method='pad'*, *axis=-2*, *order=-85*)

Expand a tensor along a specific axis to a specific size, ensuring it is at least the specified size. If the tensor is smaller than the target size, it will be padded with zeros. If the tensor is larger than the target size, it will not be cropped.

Parameters:

- **size** (`int`) – The target size.
- **method** (`str`) – The method to use in ["pad", "replicate"]. Defaults to "pad".
- **axis** (`int`) – The axis along which to expand. Defaults to -2.
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -85.

class autrainer.transforms.**FeatureExtractor**(*fe_type=None*, *fe_transfer=None*, *sampling_rate=16000*, *order=-80*)

Extract features from an audio signal using a feature extractor from the Hugging Face Transformers library.

Parameters:

- **fe_type** (`Optional`[`str`]) – The class of feature extractor to use in ["AST", "Whisper", "W2V2", None]. If None, the AutoFeatureExtractor will be used. Defaults to None.
- **fe_transfer** (`Optional`[`str`]) – The name of a pretrained feature extractor to use. If None, the feature extractor will be initialized with default values. Defaults to None.
- **sampling_rate** (`int`) – The sampling rate of the audio signal. Defaults to 16000.
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -80.

Raises:

ValueError – If neither 'fe_type' nor 'fe_transfer' is provided.

class autrainer.transforms.**GrayscaleToRGB**(*order=100*)

Convert a grayscale image to an RGB image.

Parameters:

order (`int`) – The order of the transform in the pipeline. Defaults to 100.

class autrainer.transforms.**ImageToFloat**(*order=90*)

Transform a uint8 image in the range [0, 255] to a float32 image in the range [0.0, 1.0].

Parameters:

order (`int`) – The order of the transform in the pipeline. Defaults to 90.

class autrainer.transforms.**Normalize**(*mean*, *std*, *order=95*)

Normalize a tensor with a mean and standard deviation.

Parameters:

- **mean** (`List`[`float`]) – The mean to use for normalization.
- **std** (`List`[`float`]) – The standard deviation to use for normalization.
- **order** (`int`) – The order of the transform in the pipeline. Defaults to 95.

class autrainer.transforms.**NumpyToTensor**(*order=-100*)

Convert a numpy array to a torch tensor.

Parameters:

order (`int`) – The order of the transform in the pipeline. Defaults to -100.

class autrainer.transforms.OpenSMILE(*feature_set*, *sample_rate*, *functionals=False*, *order=-80*)

Extract features from an audio signal using openSMILE.

The openSMILE library must be installed to use this transform. To install the required extras, run: 'pip install autrainer[opensmile]'.

Parameters:

- **feature_set** (`str`) – The feature set to use.
- **sample_rate** (`int`) – The sample rate of the audio signal.
- **functionals** (`bool`) – Whether to use functionals. Defaults to False.
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -80.

Raises:

ImportError – If openSMILE is not available.

class autrainer.transforms.PannMel(*window_size*, *hop_size*, *sample_rate*, *fmin*, *fmax*, *mel_bins*, *ref*, *amin*, *top_db*, *order=-90*)

Create a log-Mel spectrogram from an audio signal analogous to the Pretrained Audio Neural Networks (PANN) implementation.

For more information, see: https://doi.org/10.48550/arXiv.1912.10211

Parameters:

- **window_size** (`int`) – The size of the window.
- **hop_size** (`int`) – Hop length.
- **sample_rate** (`int`) – The sample rate of the audio signal.
- **fmin** (`int`) – The minimum frequency.
- **fmax** (`int`) – The maximum frequency.
- **mel_bins** (`int`) – The number of mel bins.
- **ref** (`float`) – The reference amplitude.
- **amin** (`float`) – The minimum amplitude.
- **top_db** (`int`) – The top decibel.
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -90.

class autrainer.transforms.RandomCrop(*size*, *method='pad'*, *axis=-2*, *fix_randomization=False*, *order=-90*)

Randomly crop a tensor along a specific axis to a specific size, ensuring it is the specified size.

If the tensor is larger than the target size, it will be randomly cropped along the specified axis.

If the tensor is smaller than the target size, it will be padded.

Parameters:

- **size** (`int`) – The target size.

- **method** (`str`) – Padding method to use if the tensor is smaller than the target size in ["pad", "replicate"]. Defaults to "pad".
- **axis** (`int`) – The axis along which to crop. Defaults to -2.
- **fix_randomization** (`bool`) – Whether to fix the randomization. Defaults to False.
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -90.

class autrainer.transforms.Resample(*current_sr*, *target_sr*, *order=-95*)

Resample an audio signal to a target sample rate.

Parameters:

- **current_sr** (`int`) – The current sample rate.
- **target_sr** (`int`) – The target sample rate.
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -95.

class autrainer.transforms.Resize(*height=64*, *width=64*, *interpolation='bilinear'*, *antialias=True*, *order=-90*)

Resize an image to a specific height and width. If "Any" is provided for the height or width, the other dimension will be used as the size, creating a square image.

Parameters:

- **height** (`int`) – The target height. If set to "Any", the width will be used as the size. Defaults to 64.
- **width** (`int`) – The target width. If set to "Any", the height will be used as the size. Defaults to 64.
- **interpolation** (`str`) – The interpolation method to use. Defaults to 'bilinear'.
- **antialias** (`bool`) – Whether to use antialiasing. Defaults to True.
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -90.

class autrainer.transforms.RGBAToRGB(*order=-95*)

Abstract class for a transform.

Parameters:

- **order** (`int`) – The order of the transform in the pipeline. Larger means later in the pipeline. If multiple transforms have the same order, they are applied in the order they were added to the pipeline. Defaults to 0.
- **kwargs** – Additional keyword arguments to store in the object.

class autrainer.transforms.RGBToGrayscale(*order=100*)

Convert an RGB or RGBA image to a grayscale image.

For the conversion to grayscale, the luminance is calculated in line with the implementation in torchvision.transforms.Grayscale: Y = 0.2989 * R + 0.587 * G + 0.114 * B.

Parameters:

order (`int`) – The order of the transform in the pipeline. Defaults to 100.

class autrainer.transforms.ScaleRange(*range=[0.0, 1.0]*, *order=90*)

Scale a tensor to a specific target range.

Parameters:

- **range** (`List[float]`) – The range to scale the tensor to. Defaults to [0.0, 1.0].
- **order** (`int`) – The order of the transform in the pipeline. Defaults to 90.

class autrainer.transforms.SpectToImage(*height*, *width*, *cmap='magma'*, *order=-90*)

Convert a spectrogram in the range [0, 1] to a 3-channel uint8 image in the range [0, 255] using a specific colormap.

Note: The transform should be used in combination with *~autrainer.transforms.ImageToFloat* to convert the image back to the range [0, 1] for training.

Parameters:

- **height** (`int`) – The height of the image.
- **width** (`int`) – The width of the image.
- **cmap** (`str`) – The colormap to use. Defaults to "magma".
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -90.

class autrainer.transforms.SquarePadCrop(*mode='pad'*, *order=-91*)

Pad or crop an image to make it square. If the image is padded, the padding will be added to the shorter sides as black pixels.

Parameters:

- **mode** (`str`) – The mode to use in ["pad", "crop"]. Defaults to "pad".
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -91.

Raises:

ValueError – If an invalid mode is provided.

class autrainer.transforms.StereoToMono(*order=-95*)

Convert a stereo audio signal to mono by taking the mean of the first dimension.

Parameters:

order (`int`) – The order of the transform in the pipeline. Defaults to -95.

class autrainer.transforms.Standardizer(*mean=None*, *std=None*, *subset='train'*, *order=-99*)

Standardize a dataset by calculating the mean and standard deviation of the data and applying the normalization.

The mean and standard deviation are automatically calculated from the data in the specified subset.

Note: The transform is applied at the specified order in the pipeline such that any preceding transforms are applied before the mean and standard deviation are calculated.

Parameters:

- **mean** (`Optional[List[float]]`) – The mean to use for normalization. If None, the mean will be calculated from the data. Defaults to None.
- **std** (`Optional[List[float]]`) – The standard deviation to use for normalization. If None, the standard deviation will be calculated from the data. Defaults to None.
- **subset** (`str`) – The subset to use for calculating the mean and standard deviation. Must be one of ["train", "dev", "test"]. Defaults to "train".
- **order** (`int`) – The order of the transform in the pipeline. Defaults to -99.

Reproducibility Guide

To ensure reproducibility of your results, the following guidelines should be followed. The goal is to allow others to reproduce the results of the project on any system by providing all necessary configurations and source code.

Project Structure

The project should be structured in a way that makes it easy to publish and reproduce the results. Therefore the following project structure is recommended:

- `conf/`: Directory for storing configuration files.
- `src/`: Directory for storing additional source code. Additional source code can also be stored in the root directory of the project.
- `requirements.txt`: File containing the pip requirements.
- `data/`: Directory for storing data files (optional).
- `results/`: Directory for storing results (optional).

To get started and create the initial project structure, the following configuration management CLI command can be used:

```
autrainer create --empty
```

Alternatively, use the following configuration management CLI wrapper function:

```
autrainer.cli.create(empty=True)
```

This will create an empty project structure only including the `conf/config.yaml` configuration file with default values.

Configuration Files

All configuration files (not provided by *autrainer* or overridden) should be stored in the `conf/` directory and published alongside the `conf/config.yaml` file. To ensure reproducibility on any system, it is recommended to solely use relative paths in the configuration files.

Custom configuration files (e.g., custom models or datasets) should be placed in the `conf/` directory. The implementation of these configurations or further processing scripts should be placed in the `src/` directory or in the root directory of the project.

In addition to configurations, the pip requirements should be stored in a `requirements.txt` file in the root directory of the project.

```
pip freeze > requirements.txt
```

Contributing

Contributions are welcome! If you would like to contribute to *autrainer*, please open an issue or a pull request on the GitHub repository.

Installation

To install *autrainer*, first ensure that PyTorch (along with torchvision and torchaudio) version 2.0 or higher is installed. For

To install *autrainer*, first ensure that PyTorch (along with torchvision and torchaudio) version 2.0 or higher is installed. For installation instructions, refer to the PyTorch website.

It is recommended to install *autrainer* within a virtual environment. To create a new virtual environment, refer to the Python venv documentation.

To set up *autrainer* for development, start by cloning the repository and then install the development dependencies with *poetry*:

```
git clone git@github.com:autrainer/autrainer.git
cd autrainer
poetry install --all-extras
```

Conventions

We use Ruff to enforce code style and formatting. Common spelling errors are automatically corrected by codespell.

Linting, formatting, and spelling checks are run with pre-commit. To install the pre-commit hooks, run the following command:

```
pre-commit install
```

Tests

We use pytest for testing. To run the tests, use the following command:

```
pytest tests
```

Citing *autrainer*

If you use *autrainer* in your research, please cite our work to acknowledge the contribution of the toolkit to your research.

Publication

The formal publication describing *autrainer* will be available soon.

Authors

autrainer is developed and maintained by the following authors:

- **Simon Rampp** simon.rampp@tum.de
- **Andreas Triantafyllopoulos** andreas.triantafyllopoulos@tum.de
- **Manuel Milling** manuel.milling@tum.de
- **Björn W. Schuller** schuller@tum.de

For any inquiries, feel free to contact us via email.